QualiAdept

Sesiunea 5 - JavaScript Asincron

Capitolul 1: Sincron vs. Asincron și Arhitectura Event Loop 2
Obiective de învățare 3
1.1 Natura JavaScript-ului: Single-Threaded (Execuția Sincronă) 3
1.2 Salvarea: Execuția Asincronă (Non-Blocking) 5
1.3 Superputerile ascunse: Web API-urile 7
1.4 Arhitectura Internă: Cum funcționează Event Loop-ul? 7
1.5 🤖 Mentalitatea de Inginer în Era AI: Conexiunea cu Playwright 9
Capitolul 2: Gestionarea Asincronismului - De la "Callback Hell" la Promisiuni și async/await 10
Obiective de învățare 10
2.1 Era Întunecată a JavaScript-ului: "Iadul Callback-urilor" (Callback Hell) 10
2.2 Salvarea: Ce este o Promisiune (Promise) și cine a inventat-o? 13
2.3 Cele 3 Stări Stricte ale unei Promisiuni 13
2.4 Revoluția din 2017: Sintaxa async și await 15
2.5 Știați că...? 16
2.6 🧠 Întrebare / Challenge de Interviu Tehnic 16
2.7 🤖 Mentalitatea de Inginer în Era AI: "Unhandled Promise Rejection" 17
Capitolul 3: Comunicarea Client-Server și Fetch API (Aducerea datelor de pe rețea) 18
Obiective de învățare 19
3.1 Arhitectura Client-Server și "Chelnerul" (API-ul) 19
3.2 Metoda fetch() și butonul de "Pauză" (await) 20
3.3 Extragerea datelor: De ce avem nevoie de un al doilea await? 21
3.4 Plasa de siguranță: Mecanica blocului try...catch 22
3.5 Știați că...? 23
3.6 🧠 Challenge de Interviu / Execuție Practică 24
3.7 🤖 Mentalitatea de Inginer în Era AI: "Nu testa orb, testează complet" 24
Capitolul 4: Concepte Avansate (Bonus) - Dincolo de GET: Trimiterea datelor (Metoda POST) 25

Obiective de învățare 25
4.1 Când "cititul" nu este de ajuns (GET vs. POST) 26
4.2 Configurarea lui fetch pentru trimiterea de date 26
4.3 Exemplu Practic: Crearea unui Task nou pe Server 27
4.4 🤖 Mentalitatea de Inginer în Era AI: Setup-ul prin API 28
Capitolul 5: Tema Sesiunii 5 & Extra Practice (Spre nivelul de Senior) 29
5.1 Tema pentru Acasă (Proiect Sesiunea 5) 29
5.2 Provocarea Bonus (Opțional): Efectul de "Loading..." și Siguranța (try/catch) 30
5.3 🔑 Cheile de Rezolvare (Soluții & Explicații) 31
Manual de Live Coding: Sesiunea 5 - JavaScript Asincron & Fetch API 33
Partea 1: Terenul de Joacă (UI Sandbox) 33
Partea 2: Exercițiile Live 35
🔴 Pentru Capitolul 1: Demonstrația Blocajului (Sincron vs Asincron) 35
🔴 Pentru Capitolul 2: Promisiuni & async/await 37
🔴 Pentru Capitolul 3: Prima cerere pe Rețea (Fetch GET) 38
🔴 Pentru Capitolul 4 (Bonus): Trimiterea Datelor pe Server (Fetch POST) 39
🔴 Pentru Capitolul 5: End-to-End & Eșec Controlat (try/catch) 41
Workbook Avansat: Sesiunea 5 - JavaScript Asincron (Extra Practice) 43
Partea 1: Setup-ul Aplicației (Sandbox-ul) 44
Partea 2: Suita de Exerciții (Executate în Consola F12) 45
💻 Exercițiul 1: Coșmarul Sincron (Event Loop Blocked) 46
💻 Exercițiul 2: Înlănțuirea Asincronă (setTimeout) 46
💻 Exercițiul 3: Crearea unei Promisiuni de la Zero 47
💻 Exercițiul 4: Consumarea Promisiunii (async/await) 48
💻 Exercițiul 5: Fetch GET - Aducerea Datelor Reale 49
💻 Exercițiul 6: Fetch POST - Raportarea unui Bug pe rețea 51
💻 Exercițiul 7: Reziliența cu Try/Catch 52
💻 Exercițiul 8: Concept Avansat - Paralelizarea (Promise.all) 54

Capitolul 1: Sincron vs. Asincron și Arhitectura Event Loop

Obiective de învățare

🎯

- Înțelegi diferența fundamentală dintre execuția Sincronă (blocantă) și Asincronă (neblocantă) prin exemple din viața reală.
- Descoperi ce sunt Web API-urile, de ce au apărut și cum oferă "superputeri" limbajului JavaScript.
- Vizualizezi mecanismul intern al browserului: Call Stack, Callback Queue și Event Loop.
- Corelezi teoria asincronă cu motivul pentru care testele Playwright au nevoie constantă de cuvântul cheie await.

1.1 Natura JavaScript-ului: Single-Threaded (Execuția Sincronă)

"Din fabrică", JavaScript este un limbaj **Single-Threaded** (cu un singur fir de execuție). Asta înseamnă că are un singur procesor principal care poate executa **o singură linie de cod la un moment dat**. Codul este citit de sus în jos, secvențial. Aceasta este **Execuția Sincronă (Blocantă)**.

Exemplul 1: Cafeneaua Sincronă

📝 Imaginează-ți o cafenea cu un singur barista. Acesta preia comanda ta (un Frappuccino complicat care durează 5 minute). În loc să te pună să aștepți la o masă și să preia comanda următorului client, barista se blochează: stă și prepară băutura ta, timp în care o coadă uriașă se formează la ușă. Nimeni altcineva nu este servit.



Fig. 1.1 - Diagrama de mai sus ilustrează perfect conceptul de **execuție sincronă și blocantă**. În acest scenariu, operațiunile se desfășoară strict una după alta:

- **Problema:** Clientul 2 este complet blocat și trebuie să aștepte ca întregul proces al Clientului 1 (inclusiv cele 5 minute de preparare) să se finalizeze înainte de a putea măcar să plaseze comanda.
- **Analogie Web:** În dezvoltarea web, dacă scriptul execută o sarcină grea în mod sincron (cum ar fi descărcarea unei imagini uriașe timp de 10 secunde), firul principal de execuție (*main thread*) este ocupat exclusiv cu acea sarcină. Drept urmare, pagina „îngheață”: browserul nu mai poate procesa acțiunile utilizatorului (click, scroll, selectare text), oferind o experiență frustrantă și catastrofală.

> 📝În web, dacă o linie de cod durează 10 secunde să descarce o imagine uriașă în mod Sincron, întreaga pagină web "îngheață" 10 secunde. Nu poți da click, nu poți face scroll. Este un scenariu catastrofal pentru experiența utilizatorului.

1.2 Salvarea: Execuția Asincronă (Non-Blocking)

Pentru a nu bloca pagina, JavaScript folosește **Asincronismul**. Acesta permite inițierea unei acțiuni lente, delegarea ei către altcineva și continuarea imediată a execuției restului de cod.

Exemplul 2: Comanda de Pizza (Viața Reală)

> 🧠 E vineri seara, vrei să faci curat în casă, dar ți-e și foame. Ai de ales cum procedezi:
> - **Abordarea Sincronă (Greșită):** Suni la pizzerie, comanzi pizza. Apoi, te duci la ușă și stai acolo, nemișcat, așteptând 40 de minute livratorul. Când vine, mănânci, și abia apoi te apuci să dai cu aspiratorul. Ai pierdut 40 de minute degeaba!
> - **Abordarea Asincronă (Smart - Cum face JS):** Suni la pizzerie și comanzi (inițiezi acțiunea). Oferi numărul tău de telefon și adresa (setezi un *Callback*). Închizi telefonul și te apuci imediat să dai cu aspiratorul (codul continuă să ruleze, nu ești blocat). Când pizza este gata, sună soneria la ușă (apare un *Eveniment*). Pui aspiratorul pe pauză, iei pizza, o mănânci (execuți *Callback-ul*), apoi revii la curățenie.

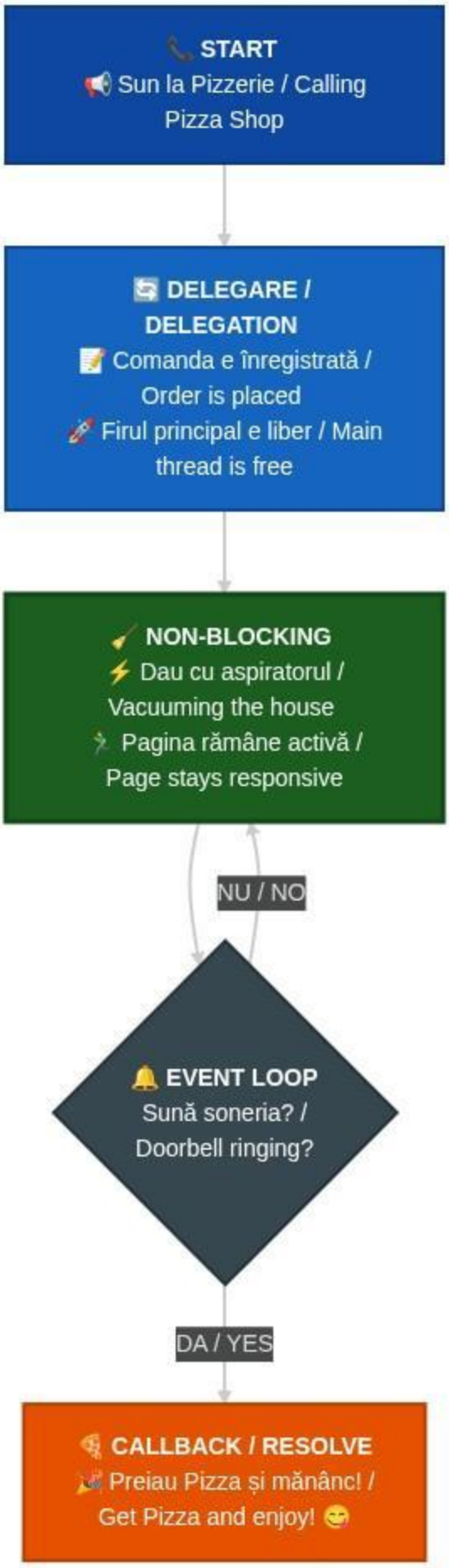

📞 START
📢 Sun la Pizzerie / Calling Pizza Shop
🔄 DELEGARE / DELEGATION
📝 Comanda e înregistrată / Order is placed
🚀 Firul principal e liber / Main thread is free
🧹 NON-BLOCKING
⚡ Dau cu aspiratorul / Vacuuming the house
🏃 Pagina rămâne activă / Page stays responsive
NU / NO
🔔 EVENT LOOP
Sună soneria? / Doorbell ringing?
DA / YES
🍕 CALLBACK / RESOLVE
🎉 Preiau Pizza și mănânc! / Get Pizza and enjoy! 😋

Fig. 1.2 - Diagrama de mai sus ilustrează funcționarea unui model **asincron** bazat pe evenimente, opusul total al scenariului blocant de la cafenea:

- **Problema Rezolvată:** Nu mai stai degeaba la telefon sau la ușă așteptând să se coacă pizza. Sarcina grea (prepararea și livrarea) a fost delegată în fundal.
- **Analogie Web:** În dezvoltarea web, aceasta este esența programării asincrone. Când pornești o acțiune de durată (cum ar fi descărcarea unei imagini sau un apel către o bază de date), browserul nu „îngheață". Firul principal de execuție lasă procesul în fundal și rămâne liber să răspundă imediat la acțiunile utilizatorului (scroll, click, animații) în timp ce „dă cu aspiratorul". Când imaginea s-a descărcat, se declanșează un eveniment (sună soneria) care afișează rezultatul pe ecran.

1.3 Superputerile ascunse: Web API-urile

Dacă **JavaScript** are **un** singur fir de execuție, cum poate **face** mai **multe lucruri** în **același** timp (cum ar fi să numere 3 secunde cu setTimeout și simultan să asculte un click pe buton)?

Secretul este că **JavaScript NU le face!** JS în sine este un limbaj limitat, care știe doar matematică, variabile și funcții. Nu știe să comunice cu internetul, nu știe să manipuleze DOM-ul și nu are ceas intern.

> 🧠 Cine face munca grea? **Mediul în care rulează JS (Browserul sau Node.js).**

Browserul (Chrome, Firefox) este un program complex scris în C++, care are sute de "fire de execuție" (threads). Browserul îi oferă lui JavaScript acces la o serie de instrumente numite **Web APIs** (Application Programming Interfaces).

- **Când au apărut?** În anii 2000, Microsoft a introdus primul API pentru a face cereri de rețea în fundal (XMLHttpRequest). Acest lucru a dus la explozia erei **AJAX** (Web 2.0). Google Maps și Gmail au fost printre primele care au folosit aceste API-uri asincrone: puteai trage de hartă și noi imagini se descărcau instant, fără ca pagina să își dea refresh!
- **Exemple de Web APIs:** setTimeout (Cronometrul browserului), document.querySelector (DOM API), fetch (Rețea/Internet), Geolocation, LocalStorage.

1.4 Arhitectura Internă: Cum funcționează Event

Loop-ul?

Sistemul care orchestrează comunicarea perfectă dintre codul tău JS și Web API-urile browserului este format din 4 componente majore. Urmărește cu atenție diagrama:



Fig. 1.3 - Diagrama de mai sus arată arhitectura exactă care permite JavaScript-ului să ruleze sarcini grele fără să blocheze interfața site-ului:

- **Call Stack (Stiva de Execuție):** Locul unde JS rulează codul curent, sincron. Funcționează pe principiul LIFO (Last In, First Out).
- **Web APIs:** Când codul tău dă de o acțiune asincronă, JS o "pasează" browserului. Browserul pornește propriul său cronometru sau cerere de rețea și eliberează instantaneu Call Stack-ul, lăsând JS-ul să ruleze mai departe.
- **Callback Queue (Coada de așteptare):** Când browserul termină de descărcat datele din rețea, el NU întrerupe codul JS care rulează în acel moment. Pune răspunsul și funcția ta într-o "coadă" (pe principiul FIFO - First In, First Out).
- **Event Loop (Bucla de Evenimente):** Este un proces continuu (un *while loop* infinit) care face o singură meserie: Se uită la Call Stack și la Callback Queue. **Regula de aur:** Event Loop-ul va muta o sarcină din Coadă înapoi în Call Stack pentru a fi procesată DOAR în secunda în care Call Stack-ul este complet GOL.

1.5 🤖 Mentalitatea de Inginer în Era AI: Conexiunea cu Playwright

🌓 **QA vs. AI: De ce eșuează testele fără asincronism?**

- **Ce se întâmplă în realitate:** Începătorii (și uneori codul generat greșit de AI) scriu comenzi de UI testate "orbește":
  1. page.click('#buton-login')
  2. expect(mesajEroare).toBeVisible()
- **Problema:** Un AI fără context de framework va trata codul de mai sus Sincron. Dar interacțiunea cu un browser real prin rețea este Asincronă (Web API). Când JS execută linia 1, o deleagă către browser și trece *instantaneu*, în zero milisecunde, la linia 2. Evident, mesajul de eroare încă nu a apărut pe ecran (pentru că request-ul abia a plecat spre server), iar testul pică fals.
- **Cum gândește un inginer QA:** Inginerul cunoaște Event Loop-ul. El știe că trebuie să țină JS-ul "în frâu". Folosind sintaxa modernă Asincronă (async / await), el spune codului de

> test: *"Așteaptă (await)! Trimite comanda de click către Web API, dar pune pauză la Call Stack-ul meu de testare până când Event Loop-ul raportează că browserul a terminat de procesat acțiunea. Abia apoi mergi la linia 2."*

Capitolul 2: Gestionarea Asincronismului - De la "Callback Hell" la Promisiuni și async/await

Obiective de învățare

> 🎯
>
> - Înțelegi limitările vechiului sistem asincron și recunoști "Iadul Callback-urilor" (Callback Hell).
> - Descoperi ce este cu adevărat o "Promisiune" (Promise), cine a standardizat-o și de ce.
> - Înveți cele 3 stări stricte ale unei Promisiuni și motivele pentru care nu pot exista mai multe.
> - Faci trecerea de la lanțurile greoaie de .then() la sintaxa modernă și curată async / await.
> - Răspunzi cu succes la scenarii clasice de interviu tehnic pe partea de asincronism.

2.1 Era Întunecată a JavaScript-ului: "Iadul Callback-urilor" (Callback Hell)

În capitolul anterior am văzut că funcțiile asincrone (precum o cerere de rețea pentru a aduce date) nu returnează rezultatul instantaneu. Pentru a ști când s-a terminat acțiunea, în primii 15 ani de existență a limbajului JavaScript, singura soluție era să trimitem o funcție (un *Callback*) ca parametru.

> 📈 *"Când termini de descărcat fisierul, te rog rulează această funcție callback".*

> Suna simplu, până când acțiunile deveneau dependente una de cealaltă. Să presupunem că avem un scenariu E2E: (1) Autentifică utilizatorul -> (2) Găsește ID-ul coșului său -> (3) Adaugă un produs în coș. Toate sunt acțiuni de rețea lente. Iată cum arăta codul:

```
// Exemplu clasic de Callback Hell (Piramida Dezastrului)
autentificareUtilizator('testUser', 'parola123', function(eroare,
utilizator) {
    if (eroare) {
        console.error("Eroare la login");
    } else {
        gasesteCos(utilizator.id, function(eroareCos, cos) {
            if (eroareCos) {
                console.error("Nu am gasit cosul");
            } else {
                adaugaProdus(cos.id, 'Telefon', function(eroareProdus,
rezultat) {
                    if (eroareProdus) {
                        console.error("Eroare la adaugare");
                    } else {
                        console.log("Produs adaugat cu succes!");
                        // ... și codul o lua tot mai mult spre dreapta
                    }
                });
            }
        });
    }
});
```

Problema: Acest cod capătă forma unei piramide orizontale (>). Este supranumit **"Callback Hell"** sau *"Pyramid of Doom"*. Este imposibil de citit, extrem de greu de menținut, iar gestionarea erorilor (acel repetitiv if (eroare)) trebuia făcută manual la fiecare pas.

Iată cum arată vizual această povară arhitecturală (observă cum fiecare pas de succes te "scufundă" și mai adânc, în timp ce trebuie să tratezi separat fiecare posibilă eroare):

QualiAdept

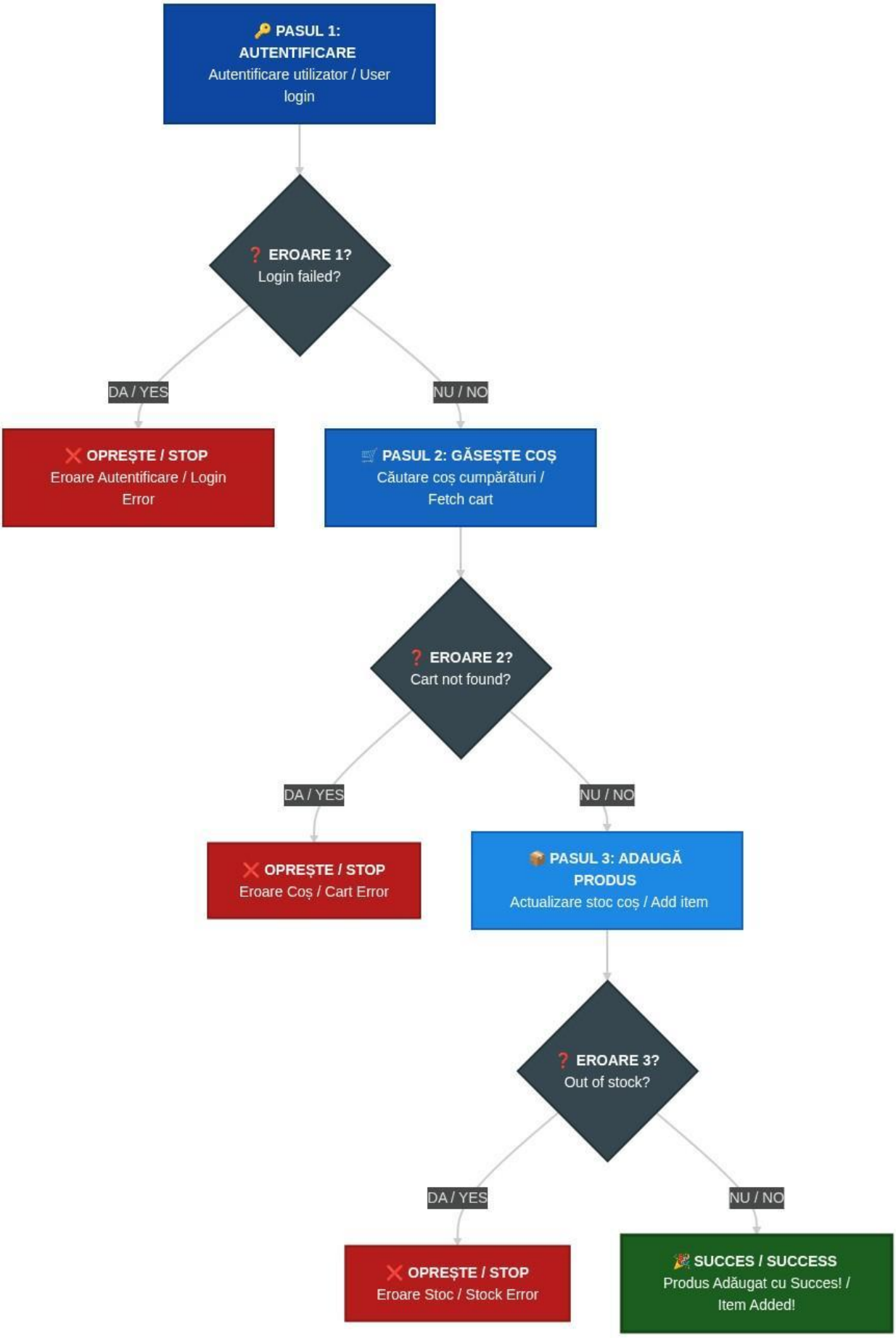

🔑 PASUL 1: AUTENTIFICARE
Autentificare utilizator / User login
❓ EROARE 1?
Login failed?
DA / YES
NU / NO
❌ OPREȘTE / STOP
Eroare Autentificare / Login Error
🛒 PASUL 2: GĂSEȘTE COȘ
Căutare coș cumpărături / Fetch cart
❓ EROARE 2?
Cart not found?
DA / YES
NU / NO
❌ OPREȘTE / STOP
Eroare Coș / Cart Error
📦 PASUL 3: ADAUGĂ PRODUS
Actualizare stoc coș / Add item
❓ EROARE 3?
Out of stock?
DA / YES
NU / NO
❌ OPREȘTE / STOP
Eroare Stoc / Stock Error
🎉 SUCCES / SUCCESS
Produs Adăugat cu Succes! / Item Added!

Fig. 2.1 - Diagrama de mai sus exemplifică vizual coșmarul arhitectural cunoscut sub numele de **Callback Hell** sau **Piramida Pierzaniei**:

- **Problema Vizuală:** Fiecare operațiune asincronă de succes te obligă să deschizi o nouă funcție imbricată (nested). Codul se extinde masiv spre dreapta, devenind imposibil de urmărit cu privirea.
- **Problema de Mentenanță:** Gestionarea erorilor este un coșmar manual. La fiecare pas individual, ești obligat să scrii o ramificație if (error) dedicată. Dacă uiți să tratezi o singură eroare la pasul 2, întreaga aplicație se poate prăbuși în tăcere, fără ca utilizatorul să știe ce s-a întâmplat.

2.2 Salvarea: Ce este o Promisiune (Promise) și cine a inventat-o?

> 💰 Pentru a rezolva acest haos, comunitatea open-source a creat un standard numit **"Promises/A+ specification"**. Văzând cât de bine funcționează, comitetul **TC39** (grupul de arhitecți care decide viitorul limbajului JavaScript) a preluat ideea și a introdus oficial obiectul **Promise** în versiunea ES6 (anul 2015).

Exemplu din Viața Reală: Comanda online și AWB-ul

O Promisiune este un **obiect special** în **JavaScript** care acționează ca un "**bilet de așteptare**" (un **proxy**) pentru o valoare care **nu** se **cunoaște** în momentul creării.

***Analogie**:* Când comanzi un laptop scump de pe eMAG, nu primești laptopul instant prin ecran. În schimb, primești un număr de urmărire (AWB) și un email care îți spune: *"Îți promitem că îți vom livra produsul"*. Tu îți poți vedea de viață mai departe. Acel AWB este echivalentul unei *Promisiuni*. AWB-ul promite că la un moment dat îți va oferi un rezultat: fie curierul ajunge cu laptopul la ușă, fie ești notificat că pachetul a fost pierdut.

2.3 Cele 3 Stări Stricte ale unei Promisiuni

Standardul **TC39** a decis un lucru critic: pentru a garanta predictibilitatea sistemelor, o Promisiune poate trece prin doar **3 stări**, și odată ce s-a finalizat, starea ei devine ***imutabilă*** (nu se mai poate

schimba niciodată).

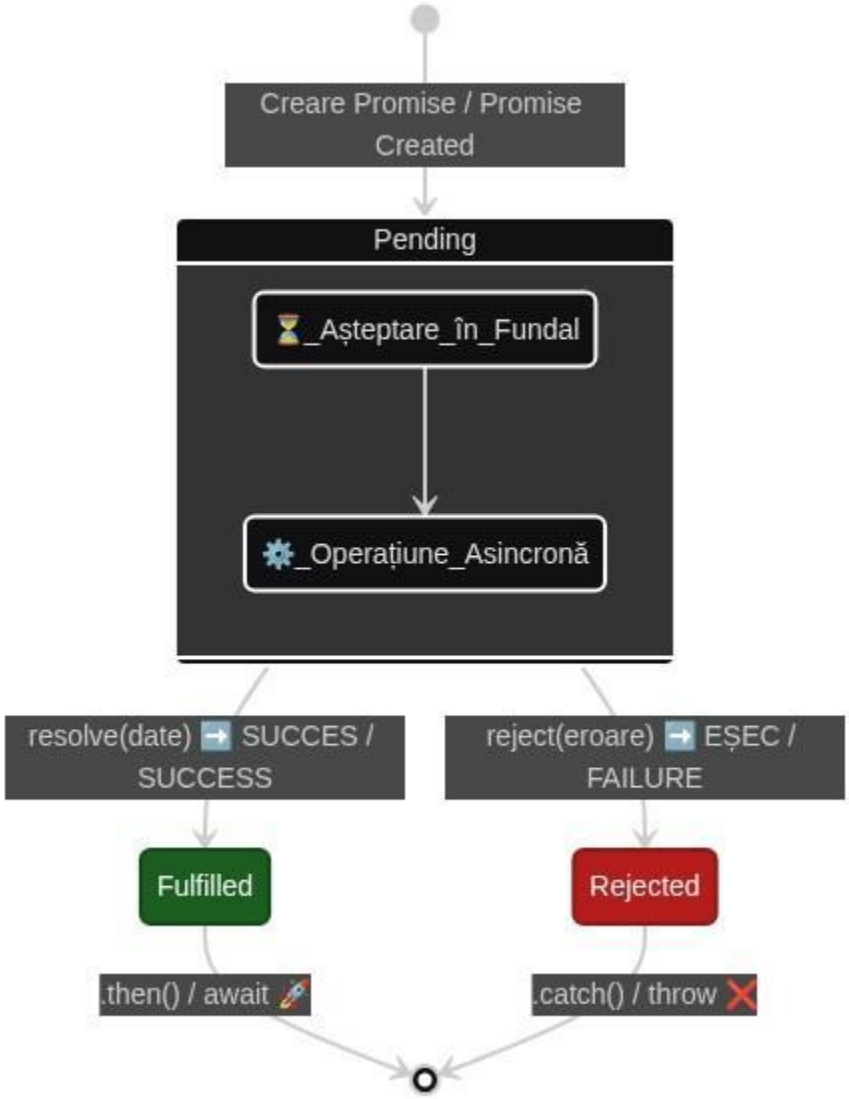


Fig. 2.2 - Cele 3 Stări Sacre ale unui Promise (TC39)

Pentru a garanta predictibilitatea totală a sistemelor web, standardul TC39 a impus o regulă de aur: starea unei promisiuni este **imutabilă** (odată ce s-a schimbat, rămâne bătută în cuie pentru totdeauna).

- **Pending (În Așteptare):** Starea inițială. Ai plasat comanda, dar curierul încă este pe drum. Cererea de rețea a plecat spre server, dar rezultatul este necunoscut.
- **Fulfilled / Resolved (Îndeplinită cu Succes):** Acțiunea s-a terminat bine. Curierul a ajuns cu pachetul. Serverul a răspuns (ex: cod 200 OK).

- **Rejected (Respinsă):** Acțiunea a eșuat. Curierul a pierdut pachetul sau a făcut accident. Serverul e picat (ex: 500 Error) sau internetul s-a întrerupt.

De ce nu există o a patra stare (ex: "Canceled" / Anulată)?

> 🧠 Arhitecții TC39 au vrut ca Promisiunile să fie sigure și deterministe. O promisiune reprezintă rezultatul *final* al unei acțiuni. Dacă ai putea "anula" o promisiune din exterior, codul tău ar putea rămâne blocat într-o stare incertă, iar funcțiile care așteptau rezultatul nu ar mai ști ce să facă. Așadar, ori se rezolvă, ori se respinge. Punct.

2.4 Revoluția din 2017: Sintaxa async și await

Inițial, pentru a citi o promisiune, dezvoltatorii foloseau comanda .then(). Deși a scăpat lumea de "Callback Hell", a creat lanțuri lungi de promisiuni.

> 📝 Așa că, în 2017 (standardul ES8), JS a introdus o sintaxă revoluționară care a schimbat modul în care scriem cod: **async / await**. Acesta este doar "syntactic sugar" (o formă mai estetică de scriere) deasupra promisiunilor, care ne permite să scriem cod asincron care arată fix ca unul sincron!

- **async:** Pus în fața unei funcții, garantează că acea funcție va returna automat o Promisiune. De asemenea, dă permisiunea de a folosi comanda await în interior.
- **await:** "Pune pauză" doar funcției curente. Îi spune JavaScript-ului: *"Oprește execuția pe această linie. Dă controlul înapoi browser-ului să facă altceva, și revino aici DOAR când Promisiunea din dreapta ta trece din starea Pending în Fulfilled."*

Același scenariu de mai sus (Cât de curat arată acum!):

```
// Logica este plană, ușor de citit de sus în jos!
const executaTestComplet = async () => {
    try {
        const utilizator = await autentificareUtilizator('testUser',
'parola123');
        const cos = await gasesteCos(utilizator.id);
        const rezultat = await adaugaProdus(cos.id, 'Telefon');
```

```
        console.log("Produs adaugat cu succes!");
    } catch (eroare) {
        // Un singur bloc Catch prinde orice eroare de la oricare din cei 3
pași!
        console.error("Test eșuat la unul din pași:", eroare);
    }
};
```

2.5 Știați că...?

💡

- **Promise.all() pentru execuție în paralel:** Dacă ai 3 cereri de rețea independente (ex: vrei să aduci profilul utilizatorului, lista de categorii și setările temei), nu trebuie să le aștepți pe rând (ceea ce ar dura, de exemplu, 1s + 1s + 1s = 3 secunde). Poți folosi await Promise.all([cerere1, cerere2, cerere3]) pentru a le declanșa pe toate simultan, reducând timpul total la doar 1 secundă! Este o tehnică vitală pentru accelerarea testelor.
- **Playwright și Promisiunile:** Aproape 99% din comenzile pe care le vei scrie în Playwright (ex: page.click(), page.fill(), page.goto()) returnează o Promisiune, pentru că se bazează pe rețea. De aceea vei folosi cuvântul await de zeci de ori într-un singur fișier de test.

2.6 🧠 Întrebare / Challenge de Interviu Tehnic

Aceasta este o întrebare extrem de comună la interviurile de QA Automation (nivel Mid/Senior) care testează înțelegerea Promisiunilor.

🗣️**Intervievatorul:** "Ești într-un test Playwright. Vrei să extragi textul de pe un buton și să-l afișezi în consolă. Scrii următorul cod:"

```
// Observație: page.locator().textContent() este o metodă asincronă
Playwright
const textButon = page.locator('.btn-login').textContent();
console.log(textButon);
```

❌**Întrebarea:** "Când rulezi testul, în loc să îți afișeze 'Logează-te', consola îți printează Promise { <pending> }. De ce se întâmplă asta și cum repari codul?"

📣**Răspunsul tău de inginer:**

"Afișează Promise { <pending> } pentru că interacțiunea cu browserul prin Playwright are loc prin rețea (WebSocket). Deci, metoda textContent() nu returnează textul instantaneu, ci returnează o *Promisiune* (un AWB) că va aduce textul.

Deoarece codul a uitat să folosească cuvântul cheie await, JavaScript-ul nu a pus pe pauză execuția. A luat Promisiunea creată (care în milisecunda 1 este evident în starea *Pending*), a salvat-o în variabilă și a printat-o direct pe ecran.

✅ **Pentru a repara**, trebuie să adăugăm await în fața comenzii:

const textButon = await page.locator('.btn-login').textContent(); - abia acum scriptul va aștepta ca promisiunea să devină *Fulfilled* și va pune textul real ('Logează-te') în variabilă."

2.7 🤖 Mentalitatea de Inginer în Era AI: "Unhandled Promise Rejection"

🌓 **QA vs. AI: Promisiunile Abandonate**

- **Ce face AI-ul bine:** Generatoarele de cod (Copilot, ChatGPT) știu regulile generale. Dacă îi spui să-ți scrie un test Playwright, în marea majoritate a cazurilor va pune corect async și await.
- **Capcana (Unde AI-ul eșuează):** Uneori, când îi ceri AI-ului să scrie o funcție *custom* utilitară (de exemplu o buclă for care curăță baza de date înainte de test), AI-ul ar putea scrie logica, dar ar putea "uita" să marcheze sub-comenzile cu await. În JavaScript-ul modern, dacă o Promisiune eșuează (stare: *Rejected*) și nu a fost "așteptată" (awaited) corespunzător pentru a prinde eroarea, sistemul va crăpa brusc cu faimoasa eroare **Unhandled Promise Rejection** (O promisiune a fost respinsă și nimeni nu a fost acolo să gestioneze eroarea). Acest lucru poate cauza instabilitate imensă în pipeline-ul CI/CD.
- **Cum gândește un inginer QA:** Când inginerul face Code Review pe un script generat de AI sau de un coleg Junior, ochiul lui scanează exclusiv o anumită regulă: *Avem vreo metodă de Web API apelată fără await în față?* Inginerul forțează arhitectura asincronă impecabilă pentru a asigura un flow determinist și sigur.

Capitolul 3: Comunicarea Client-Server și Fetch API (Aducerea datelor de pe rețea)

Obiective de învățare

🎯

- Înțelegi ce este un API (Application Programming Interface) și cum funcționează arhitectura Client-Server.
- Folosești metoda nativă fetch() pentru a face cereri HTTP reale către un server și înțelegi parametrii acesteia.
- Combini fetch() cu async / await pentru a "pune pe pauză" codul, a descărca și a citi date în format JSON.
- Descoperi mecanica internă a blocurilor try / catch pentru gestionarea erorilor fatale de rețea.
- Corelezi aceste concepte cu testarea de API din Playwright.

3.1 Arhitectura Client-Server și "Chelnerul" (API-ul)

Până acum, aplicația noastră a trăit doar în browser (Frontend). Dar într-o aplicație reală, lista de produse, utilizatorii și parolele nu sunt stocate în fișierul HTML, ci pe un calculator puternic aflat la distanță, numit **Server** (sau Backend/Bază de Date).

Când tu (Clientul) vrei să vezi lista de produse, browserul tău trebuie să trimită un mesaj prin internet către Server, iar Serverul trebuie să trimită datele înapoi.

Ce este un API?

API (Application Programming Interface) este puntea de legătură dintre Frontend și Backend.

Analogie de QA: Ești la restaurant. Tu ești **Clientul** (Frontend-ul/Browserul). Bucătăria este **Serverul** (Backend-ul unde se prepară datele). Tu nu ai voie să intri direct în bucătărie să îți iei mâncarea. Ai nevoie de un **Chelner**. Acel chelner este **API-ul**. El preia comanda ta (Request), o duce la bucătărie și îți aduce înapoi mâncarea (Response).

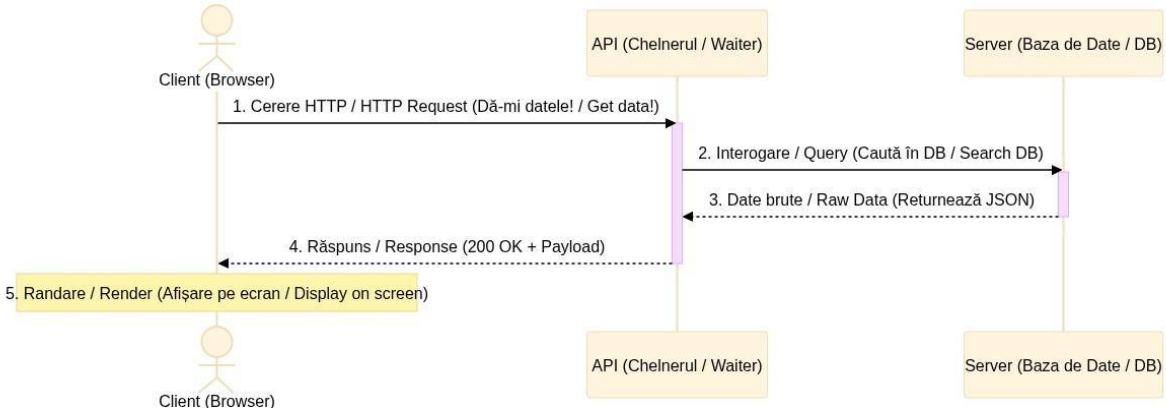


Fig. 3.1 - Diagrama de secvență de mai sus ilustrează pașii cronologici prin care trec datele din momentul în care un utilizator accesează o pagină și până când conținutul devine vizibil:

- **Cererea inițială (Pașii 1-2):** Browserul (Clientul) trimite o scrisoare digitală către API (Chelnerul). API-ul nu deține produsele fizic, așa că merge mai departe în spate, la depozit (Serverul/Baza de date), pentru a le aduce.
- **Procesarea asincronă (Pașii 3-5):** Depozitul trimite cutia cu date brute înapoi la API (sub formă de text JSON). API-ul ambalează frumos răspunsul cu o ștampilă de succes (200 OK) și îl livrează înapoi browserului. În tot acest timp, dacă procesul a fost asincron, utilizatorul a putut folosi alte părți ale site-ului. La final, browserul deschide cutia și randează (desenează) elementele vizuale pe ecran.

3.2 Metoda fetch() și butonul de "Pauză" (await)

În JavaScript-ul modern, unealta nativă (fără a instala librării externe) folosită pentru a trimite o cerere HTTP prin rețea se numește **fetch()** (în traducere: "a aduce").

Cum funcționează fetch()?

> 🧠 În cea mai simplă formă a sa, fetch() are nevoie de un singur parametru obligatoriu: **URL-ul (adresa)** de unde vrei să aduci datele. Implicit, dacă nu îi spui altceva, fetch() va face o cerere de tip **GET** (adică doar cere date, nu șterge și nu modifică nimic pe server).
> Deoarece trimiterea unui mesaj pe internet până la un server din SUA și înapoi este o acțiune lentă, metoda fetch() returnează întotdeauna o **Promisiune** (un "AWB" care îți spune că cererea a plecat).

Puterea lui await:

Aici intervine magia lui await. Fără el, JavaScript-ul ar trimite cererea și ar trece instantaneu la linia următoare, înainte ca datele să fi ajuns! Punând await în fața lui fetch(), tu apeși pe un buton de "Pauză" pe telecomandă. Îi spui codului: *"Oprește-te pe această linie! Nu face niciun pas mai departe până când Promisiunea nu se întoarce de pe internet cu starea Fulfilled (cerere reușită) sau Rejected (server picat)."*

```
// O funcție asincronă care aduce date de pe o adresă web
const aduceDateDeLaServer = async () => {
    console.log("1. Cererea a plecat spre server...");

    // Execuția funcției se OPREȘTE aici până vine răspunsul
    const raspuns = await
fetch('https://jsonplaceholder.typicode.com/users/1');

    // Această linie se execută abia DUPĂ ce răspunsul a sosit
    console.log("2. Răspunsul brut a sosit!", raspuns);
};
```

3.3 Extragerea datelor: De ce avem nevoie de un al doilea await?

Dacă rulezi codul de mai sus, vei vedea că variabila raspuns nu conține numele utilizatorului pe care îl cauți. Conține un obiect HTTP uriaș, care îți arată Statusul (ex: 200) și niște informații despre rețea.

Pentru a citi datele reale din interiorul acestui pachet, trebuie să le despachetăm. Datele vin aproape mereu în format **JSON** (JavaScript Object Notation - o structură de text care seamănă perfect cu obiectele din JS).

Pentru a transforma textul venit de pe rețea într-un obiect JS pe care să îl putem citi, folosim metoda .json().

Atenție! Pentru că pachetul de date (payload-ul) venit de la server poate avea 2 Megabytes și poate dura ceva timp să fie despachetat/tradus, comanda .json() este și ea o acțiune asincronă. Deci **returnează a doua Promisiune**, motiv pentru care are nevoie de propriul său await!

Așa se scrie corect un request HTTP complet în 2026:

```
const afiseazaUtilizator = async () => {
    // Pasul 1: Așteptăm să se termine călătoria pe rețea
    const raspunsHTTP = await
fetch('https://jsonplaceholder.typicode.com/users/1');

    // Pasul 2: Așteptăm despachetarea datelor din HTTP în obiect JS (JSON)
    const utilizator = await raspunsHTTP.json();

    // Pasul 3: Acum avem un Obiect real și putem accesa proprietățile lui!
    console.log("Numele utilizatorului este:", utilizator.name);
    console.log("Email-ul lui este:", utilizator.email);
};
```

3.4 Plasa de siguranță: Mecanica blocului try...catch

Pe internet, lucrurile merg prost frecvent. Serverul pică, utilizatorul nu are semnal la telefon sau link-ul (URL-ul) este scris greșit. Dacă o Promisiune eșuează (stare: *Rejected*) și noi nu prindem eroarea, aplicația se blochează complet (dă crash fatal).

Pentru a preveni acest lucru, programatorii folosesc un mecanism de urgență numit **try...catch**.

Cum funcționează:

- **try { ... } (Încearcă):** Aici pui tot codul tău principal, "Happy Path-ul" (calea fericită). JS încearcă să execute codul linie cu linie. Dacă totul merge bine, blocul catch este ignorat complet.
- **Crash-ul invizibil:** Dacă *absolut orice* comandă din interiorul blocului try eșuează (ex: pică internetul exact când se face fetch()), execuția codului din try **se oprește instantaneu** pe acea linie. Următoarele linii din try nu se mai execută!
- **catch (eroare) { ... } (Prinde):** Scriptul "sare" imediat în acest bloc. Variabila eroare va conține mesajul tehnic detaliat despre ce a mers prost. Aici tu, ca inginer, decizi cum gestionezi situația: afișezi un mesaj prietenos utilizatorului ("Ne pare rău, te rugăm să reîncerci") sau pici testul în mod controlat.

```
const cereDateSigure = async () => {
    try {
```

```
        console.log("A. Începem tranzacția..."); // Se execută

        // Dacă URL-ul e greșit sau pică rețeaua, această linie ARUNCĂ O
EROARE!
        const raspuns = await
fetch('https://jsonplaceholder.typicode.com/users/9999');

        // BUNA PRACTICĂ QA: Verificăm Statusul HTTP înainte să mergem mai
departe
        if (raspuns.status === 404) {
            console.error("Test Fail: Utilizatorul nu a fost găsit (Cod
404).");
            return; // Oprim funcția aici
        }

        const date = await raspuns.json();
        console.log("B. Date primite:", date); // Nu se execută dacă a fost
eroare mai sus

    } catch (eroare) {
        // Am "prins" eroarea, deci aplicația NU a dat crash!
        console.error("Test Error: A apărut o problemă severă de rețea sau
despachetare:", eroare.message);
    }
};
```

3.5 Știați că...?

💡

- **Codurile de Status HTTP (Ce trebuie să știe un QA):**
  - 200 (OK): Cererea a avut succes, ai primit datele.
  - 201 (Created): O resursă nouă a fost creată pe server (ex: ai trimis un formular valid).
  - 400 (Bad Request): Tu (Clientul) ai trimis date greșite sau incomplete.
  - 401 (Unauthorized): Nu ești logat, trebuie să trimiți un token de acces.
  - 404 (Not Found): Adresa sau resursa cerută nu există pe server.

    - 500 (Internal Server Error): Codul backend-ului a dat crash. Tu ai făcut totul bine, dezvoltatorii trebuie să repare serverul!
- **Playwright și Interceptarea de Rețea (Mocking):** În Modulul 7, vei învăța o "magie" supremă. În loc să lași comanda de testare să ceară date reale de la server, cu Playwright poți intercepta acel "Chelner" pe drum, îi pui pe tavă niște date false create de tine (Mock Data) și îl trimiți înapoi la Client! Asta te va ajuta să testezi aplicația Frontend indiferent dacă Backend-ul (Serverul) este funcțional sau nu la momentul rulării testului!

3.6 🧠 Challenge de Interviu / Execuție Practică

🧠 Deschide consola browserului tău și execută o cerere reală!

Misiunea ta:

Scrie o funcție asincronă (Arrow Function cu async) numită aducePostari.

- În interiorul ei, folosește await fetch() pentru a cere lista de postări de la acest API public: 'https://jsonplaceholder.typicode.com/posts'.
- Extrage datele folosind await raspuns.json().
- Având în vedere că răspunsul despachetat este o listă (Array), folosește comanda console.log() pentru a afișa numărul total de elemente primite (citind proprietatea .length a listei).

(Răspunsul așteptat în consolă ar trebui să fie: "Au fost primite 100 de postări.")

3.7 🤖 Mentalitatea de Inginer în Era AI: "Nu testa orb, testează complet"

🌓 QA vs. AI: Ignorarea Codurilor de Eroare

- **Ce face AI-ul bine:** Îți generează instantaneu un test API în Playwright, folosind corect comanda echivalentă a lui fetch (request.get()) și extrăgând corect JSON-ul.
- **Capcana (Unde AI-ul eșuează):** Un AI necizelat va trimite cererea și va încerca imediat să verifice dacă json.name === "Alex". Dacă serverul pică și returnează eroarea *500 Internal Server Error* (care pe rețea este de obicei o pagină HTML de eroare, nu un format JSON valid), comanda de parsare .json() va da un crash urât ("Unexpected token < in JSON at position 0"), iar în raportul de testare va apărea că testul e "Rupt" (Broken), în loc să spună clar că serverul e picat.
- **Cum gândește un inginer QA:** Inginerul se asigură că testul său acționează ca un scut. Înainte de a face "despachetarea" periculoasă a JSON-ului, el obligă scriptul să valideze tranzacția HTTP: expect(response.status()).toBe(200);. Asta asigură un raport de testare clar și profesional (testul eșuează cu un mesaj de business impecabil: "Expected 200, but got 500").
- **Locul tău pe piață:** A scrie E2E nu înseamnă doar a verifica vizual butoane care se mișcă pe ecran, ci a verifica straturile ascunse (Network). Inginerii de automatizare foarte căutați sunt cei care pot valida nu doar că "produsul a apărut în coș", ci și că "răspunsul API din spate care a generat produsul s-a executat în mai puțin de 500ms și a întors statusul 200 OK". Aceasta se numește **Testare Arhitecturală Integrată**.

Capitolul 4: Concepte Avansate (Bonus) - Dincolo de GET: Trimiterea datelor (Metoda POST)

Obiective de învățare

🎯

- Înțelegi diferența esențială dintre metodele HTTP GET (citire) și POST (creare/trimitere).
- Înveți cum să configurezi funcția fetch() cu parametri suplimentari (Method, Headers, Body).
- Transformi un Obiect JavaScript într-un text compatibil pentru rețea folosind JSON.stringify().
- Corelezi aceste concepte cu modul în care vei pregăti datele de test direct prin API în Playwright, ocolind interfața grafică.

4.1 Când "cititul" nu este de ajuns (GET vs. POST)

În capitolul precedent am trimis "Chelnerul" (API-ul) la bucătărie doar să ne aducă lista cu ce există deja (ex: lista de task-uri). Asta se numește o cerere de tip **GET** (cea implicită în fetch()).

Dar ce ne facem când utilizatorul completează formularul nostru și apasă "Adaugă Task"? Frontend-ul trebuie să ia acel text, să îl împacheteze și să îl trimită Serverului pentru a fi salvat în baza de date. Când *trimitem* date noi pentru a fi salvate, folosim metoda HTTP **POST**.

- **Alte metode importante pentru QA:**
  - PUT / PATCH: Folosite pentru a *actualiza* sau a modifica o resursă existentă (ex: schimbarea parolei).
  - DELETE: Folosită pentru a *șterge* o resursă de pe server.

4.2 Configurarea lui fetch pentru trimiterea de date

Dacă pentru GET aveam nevoie doar de URL, pentru POST trebuie să îi dăm lui fetch() un al doilea parametru: un obiect de **configurare** (opțiuni). Acest obiect îi spune serverului trei lucruri esențiale:

- **method**: Ce acțiune facem (ex: 'POST').
- **headers**: O notă atașată pachetului. Trebuie să îi spunem serverului în ce limbă îi vorbim. De obicei, îi spunem că îi trimitem date de tip JSON: { 'Content-Type': 'application/json' }.
- **body**: Datele efective pe care le trimitem. *Atenție!* Pe internet nu poți trimite direct un Obiect JavaScript. Trebuie să îl transformi într-

un string (text brut). Folosim comanda inversă față de ce am învățat anterior: **JSON.stringify(obiectulMeu)**.

4.3 Exemplu Practic: Crearea unui Task nou pe Server

Iată cum arată codul asincron complet pentru a trimite un task nou către server, printează răspunsul (de obicei un status 201 Created și ID-ul noului task).

```
const trimiteTaskCatreServer = async (numeTaskNou) => {
    try {
        // 1. Pregătim datele (Obiectul JS)
        const pachetDate = {
            title: numeTaskNou,
            completed: false,
            userId: 1
        };

        // 2. Apelăm fetch cu URL-ul și Obiectul de Configurare
        const raspuns = await
fetch('https://jsonplaceholder.typicode.com/todos', {
            method: 'POST', // Spunem că vrem să creăm ceva
            headers: {
                // Îi spunem serverului să se aștepte la format JSON
                'Content-Type': 'application/json'
            },
            // Transformăm obiectul JS în Text (Stringify)
            body: JSON.stringify(pachetDate)
        });

        // 3. Verificăm dacă salvarea a avut succes (Cod 201 înseamnă
"Creat")
        if (raspuns.status === 201) {
            const dateSalvate = await raspuns.json();
            console.log("Succes! Serverul a salvat task-ul și i-a dat ID-
ul:", dateSalvate.id);
        } else {
            console.error("A apărut o problemă la salvare. Status:",
raspuns.status);
        }
```

```
    } catch (eroare) {
        console.error("Eroare fatală de rețea la trimitere:",
eroare.message);
    }
};

// Apelăm funcția
trimiteTaskCatreServer("Învață testarea de API!");
```

4.4 🤖 Mentalitatea de Inginer în Era AI: Setup-ul prin API

> 🌓 **QA vs. AI: Interfața (UI) vs. Rețeaua (API)**
>
> - **Scenariul:** Vrei să scrii un test automatizat în Playwright care verifică funcția de **Ștergere a unui Task**.
> - **Ce va face un Junior (sau un AI banal):** Va scrie testul astfel:
>   1. Deschide pagina de adăugare task.
>   2. Completează câmpul "Task Nou" (UI).
>   3. Dă click pe butonul "Salvează" (UI).
>   4. Caută în listă butonul de "Șterge" (UI).
>   5. Dă click pe el și validează ștergerea.
> - **Problema:** Pașii 1-3 sunt doar un *Setup* (pregătirea condițiilor pentru test), dar sunt lenți (durează 5 secunde) și se pot rupe (dacă se schimbă UI-ul de adăugare, pică testul de ștergere).
> - **Cum gândește un inginer QA (Senior Automation):** Inginerul cunoaște metoda **POST**. El va ocoli complet interfața grafică lentă. La începutul testului, în cârligele de pregătire (ex: beforeAll), el va lansa un fetch('.../todos', { method: 'POST', ... }) invizibil, direct către baza de date, creând task-ul în 100 de milisecunde! Apoi, testul lui de UI se focusează exclusiv pe validarea vizuală a butonului de *Ștergere*, reducând durata testului de la 5 secunde la 1 secundă și crescând stabilitatea acestuia exponențial.

Capitolul 5: Tema Sesiunii 5 & Extra Practice (Spre nivelul de Senior)

5.1 Tema pentru Acasă (Proiect Sesiunea 5)

📖 **Sarcina ta (Task-ul de business):**

> 📝 Până acum, la deschiderea aplicației noastre "Task Tracker", lista era mereu goală (sau conținea date statice scrise direct în HTML). Într-o aplicație reală Enterprise, când utilizatorul accesează pagina, Frontend-ul face instantaneu o cerere către Backend pentru a aduce task-urile salvate în baza de date.

> Sarcina ta este să scrii o funcție care face exact acest lucru: la încărcarea paginii, va descărca 5 task-uri de la un API public și le va desena pe ecran!

🔗 ***Notă pentru execuție:*** *Vei adăuga acest cod în fișierul tău* ***app.js*** *existent.*

⚠️ **Cerințe Tehnice (Acceptance Criteria pentru validare):**

- **Crearea funcției asincrone:** Declară o Arrow Function asincronă numită incarcaTaskuriInitiale = async () => { ... }.
- **Fetch API:** În interiorul funcției, folosește comanda await fetch() pentru a face o cerere GET către următoarea adresă: 'https://jsonplaceholder.typicode.com/todos?_limit=5'. (Acest API public îți va returna o listă cu exact 5 task-uri de test).
- **Extragerea datelor:** Despachetează răspunsul primit folosind metoda .json() (nu uita de al doilea await!). Variabila ta va fi acum un Array cu 5 Obiecte.
- **Randarea în DOM (Combinare cu Sesiunea 4):**
  - Găsește lista principală din HTML (#active-tasks-list).
  - Folosește o buclă for (sau .forEach()) pentru a parcurge Array-ul primit de la server.
  - Pentru fiecare obiect din listă, creează dinamic un element <li>.
  - Setează textul elementului <li> cu titlul task-ului venit de pe rețea (proprietatea .title a obiectului JSON).
  - Injectează elementul <li> în listă folosind appendChild().
- **Apelarea funcției:** Scrie pur și simplu incarcaTaskuriInitiale(); la finalul fișierului tău app.js pentru ca funcția să se declanșeze automat imediat ce s-a încărcat scriptul.

*(**Analogie QA**: Când vei automatiza cu Playwright, exact această întârziere cauzată de fetch va necesita ca robotul tău să folosească o asertare de tip "Auto-Waiting" - ex: await expect(page.locator('.task-item')).toHaveCount(5) - pentru a-i da timp JavaScript-ului să aducă datele și să randeze cele 5 elemente pe ecran).*

5.2 Provocarea Bonus (Opțional): Efectul de "Loading..." și Siguranța (try/catch)

💣 **Scenariul:**

Ești la o companie de top. Designerul îți spune: *"Cererea către server poate dura 2 secunde.*

Dacă ecranul rămâne gol în acest timp, utilizatorul va crede că aplicația e blocată. Mai mult, dacă pică serverul, vreau să afișăm o eroare roșie, nu doar să dăm crash în consolă."

🤖**Misiunea ta:**

Modifică funcția de mai sus.

- Găsește elementul cu ID-ul #loading-msg (pe care l-am creat în Sesiunea 4).
- La prima linie din funcția ta, fă mesajul vizibil: mesajLoading.style.display = 'block';.
- Înfășoară logica de fetch și for într-un bloc **try { ... }**.
- La finalul blocului try, ascunde mesajul: mesajLoading.style.display = 'none';.
- Adaugă blocul **catch (eroare) { ... }**. Aici, dacă rețeaua pică (poți testa stricând intenționat URL-ul), prinde eroarea și afișează direct în DOM, în locul listei de task-uri, un text roșu: *"Eroare critică de rețea. Te rugăm să revii."*

5.3 🔑 Cheile de Rezolvare (Soluții & Explicații)

🚨 🛑 🚫 🚧 **Nu trișa**! Citește asta doar după ce te-ai blocat sau ai încercat să scrii codul singur. Aceasta este o problemă reală de interviu pentru pozițiile de Frontend / SDET!

✅**Soluția Completă (Cu Provocarea Bonus inclusă):**

```
// Funcția noastră principală asincronă
const incarcaTaskuriInitiale = async () => {
    const listaTaskuri = document.querySelector('#active-tasks-list');
    const mesajLoading = document.querySelector('#loading-msg');

    // 1. Arătăm utilizatorului că sistemul "gândește" (UX Design)
    if (mesajLoading) mesajLoading.style.display = 'block';

    try {
        // 2. Facem cererea de rețea (Așteptăm "Chelnerul")
        const raspuns = await
fetch('https://jsonplaceholder.typicode.com/todos?_limit=5');

        // Verificăm dacă statusul e 200 OK
        if (!raspuns.ok) throw new Error("Eroare de Server!");

        // 3. Despachetăm datele JSON
        const taskuriDeLaServer = await raspuns.json();
```

```
        // 4. Parcurgem lista și injectăm în DOM (Fix ca în Sesiunea 4)
        for (let i = 0; i < taskuriDeLaServer.length; i++) {
            const liNou = document.createElement('li');
            liNou.className = 'task-item';
            // Preluăm proprietatea 'title' din obiectul JSON returnat de
API
            liNou.textContent = "☁️ " + taskuriDeLaServer[i].title;

            listaTaskuri.appendChild(liNou);
        }

    } catch (eroare) {
        // 5. Ce facem dacă internetul e picat sau API-ul nu răspunde?
        console.error("Atenție, a picat cererea:", eroare);
        listaTaskuri.innerHTML = '<li style="color: red; font-weight:
bold;">Eroare critică de rețea. Nu am putut încărca datele.</li>';

    } finally {
        // Blocul 'finally' se execută mereu, indiferent dacă a fost Try
sau Catch!
        // Aici ascundem mereu mesajul de "Loading..." la final.
        if (mesajLoading) mesajLoading.style.display = 'none';
    }
};

// Declanșăm funcția automat la rularea scriptului
incarcaTaskuriInitiale();
```

🎉 **Felicitări! Ai finalizat Sesiunea 5!**

Privește codul de mai sus. Aceasta nu mai este doar o simplă joacă în consolă, este **Arhitectură de Software Reală**. Ai gestionat starea aplicației (Loading), ai comunicat asincron cu o bază de date (Fetch/Await), ai procesat un payload (JSON) și ai tratat fail-safe o excepție (Try/Catch). Ești gata să domini mediul web! Urmează Playwright.

Manual de Live Coding: Sesiunea 5 - JavaScript Asincron & Fetch API

> 🏫 **Ghid pentru Sesiunea Live:** Acest document conține mediul de lucru (Sandbox) și snippet-urile de cod pe care le vom scrie și rula *împreună* în timpul cursului. Fiecare exercițiu corespunde unui capitol din teorie și ne ajută să demonstrăm vizual conceptele de Asincronism, Promisiuni și Fetch API.

Partea 1: Terenul de Joacă (UI Sandbox)

📂 **Acțiune Live:** Toată clasa deschide VS Code, creează un fișier **live-session.html**, copiază codul de mai jos și îl deschide în browser (Live Server / dublu-click). Aici vom rula toate

experimentele din Sesiunea 5!

```
<!DOCTYPE html>
<html lang="ro">
<head>
    <meta charset="UTF-8">
    <title>S5 Live Sandbox</title>
    <style>
        body { font-family: 'Segoe UI', sans-serif; background-color:
#282c34; color: white; padding: 30px; }
        .container { max-width: 700px; margin: auto; background: #21252b;
padding: 20px; border-radius: 10px; box-shadow: 0 4px 10px rgba(0,0,0,0.5);
}
        button { background: #61afef; color: #282c34; border: none;
padding: 10px 15px; cursor: pointer; border-radius: 5px; font-weight: bold;
font-size: 14px; margin-right: 10px; }
        button.btn-danger { background: #e06c75; color: white; }
        .output-screen { background: #1e1e1e; color: #98c379; padding:
15px; margin-top: 20px; border-radius: 5px; font-family: 'Courier New',
Courier, monospace; min-height: 50px; border: 1px solid #3e4451; }
        ul { list-style: none; padding: 0; }
        li { background: #3e4451; margin-bottom: 8px; padding: 10px;
border-radius: 4px; border-left: 5px solid #c678dd; }
    </style>
</head>
<body>
    <div class="container">
        <h2>🚀 Modul Live: Testare API & Asincronism</h2>

        <div>
            <button id="btn-sync" class="btn-danger">Test Sincron
(Blochează UI!)</button>
            <button id="btn-async">Test Asincron (SetTimeout)</button>
        </div>

        <div id="live-console" class="output-screen">
            Aștept comenzi din consolă...
        </div>

        <ul id="live-data-list">
            <!-- Aici vom injecta datele de la server -->
        </ul>
    </div>
```

```
    <!-- Aici vom scrie codul JS live -->
    <script>
        // Variabile globale pentru a manipula interfața în exercițiile
următoare
        const ecran = document.querySelector('#live-console');
        const listaHtml = document.querySelector('#live-data-list');
    </script>
</body>
</html>
```

Partea 2: Exercițiile Live

Pe măsură ce parcurgem teoria, vom adăuga aceste funcții direct în tag-ul <script> din fișierul live-session.html și le vom apela pentru a demonstra comportamentul pe viu.

🟤 Pentru Capitolul 1: Demonstrația Blocajului (Sincron vs Asincron)

🧨 **Scenariu Live:** Vom vedea cum un "while loop" greu (sincron) face browserul să înghețe, pe când setTimeout (asincron) lasă interfața liberă.

🤯 ❓ **Cum testăm efectiv:**

- În fișierul live-session.html, fix sub linia const listaHtml = ..., adaugă codul de mai jos.
- Salvează fișierul și dă refresh la pagină în browser.
- Apasă butonul roșu **Test Sincron** și încearcă *imediat* să selectezi textul de pe pagină cu mouse-ul. Vei observa că totul a înghețat, nu poți selecta absolut nimic timp de 3 secunde! (Dacă ești pe Chrome, nici măcar textul galben nu va apuca să apară până nu se termină blocajul).
- Apoi, apasă butonul albastru **Test Asincron** și încearcă din nou să selectezi text. Interfața este perfect fluidă, deși scriptul numără 3 secunde în fundal pentru a aduce "pizza".

Cod de adăugat în tag-ul <script>:

```
const btnSync = document.querySelector('#btn-sync');
const btnAsync = document.querySelector('#btn-async');
```

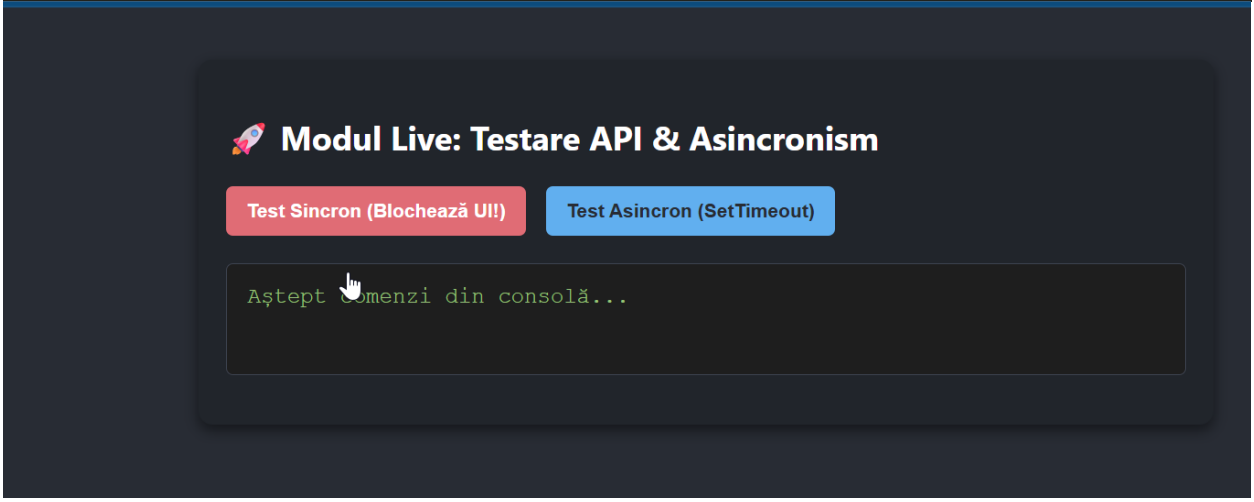
```
// 1. Abordarea Sincronă (Blocantă) - "Iadul procesării"
btnSync.addEventListener('click', () => {
    ecran.innerHTML = "<span style='color:#e06c75;'>⏳ Am blocat pagina 3 secunde! Încearcă să selectezi textul acesta...</span>";

    // Forțăm o blocare reală a firului principal (Main Thread)
    const start = Date.now();
    while(Date.now() - start < 3000) { /* Procesor la 100%, blocaj total! */ }

    ecran.innerHTML = "✅ Pagina s-a deblocat.";
});

// 2. Abordarea Asincronă (Non-blocking) - "Delegarea"
btnAsync.addEventListener('click', () => {
    ecran.innerHTML = "⏳ Cererea a plecat în fundal (Asincron). Poți selecta text, nu ești blocat!";

    // Delegăm așteptarea către Web API
    setTimeout(() => {
        ecran.innerHTML = "🍕 Pizza a ajuns (după 3 secunde din fundal)!";
    }, 3000);
});
```





🟤 Pentru Capitolul 2: Promisiuni & async/await

📌**Scenariu Live:** Vom crea propria noastră Promisiune (un AWB) care se rezolvă după 2 secunde și o vom consuma folosind sintaxa Playwright (await).

🤔 ❓ Cum testăm efectiv:

- Continuă să scrii codul de mai jos în tag-ul <script> din fișierul tău live-session.html.
- Salvează fișierul și dă refresh paginii.
- Deschide **Consola (F12)** din browser.
- Pentru a declanșa funcția, scrie în consolă comanda executaTestPromisiune() și apasă Enter.
- Privește ecranul negru UI din pagină: vei vedea imediat mesajul "Aștept pachetul (Stare: Pending)...".
- Numără 2 secunde și observă cum textul se schimbă automat în "💻 Laptopul de Gaming a fost livrat...", demonstrând vizual cum await-ul a "pus pe pauză" doar acea funcție.

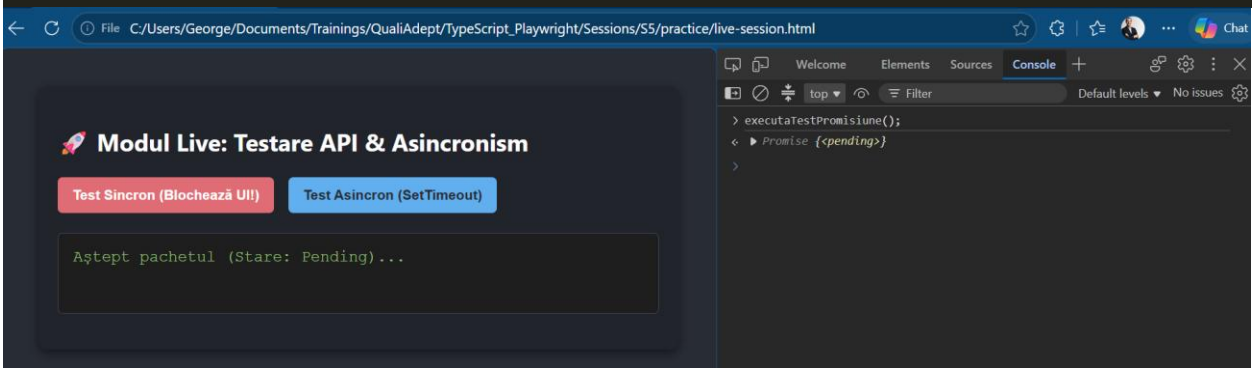
Cod de adăugat în tag-ul <script>:

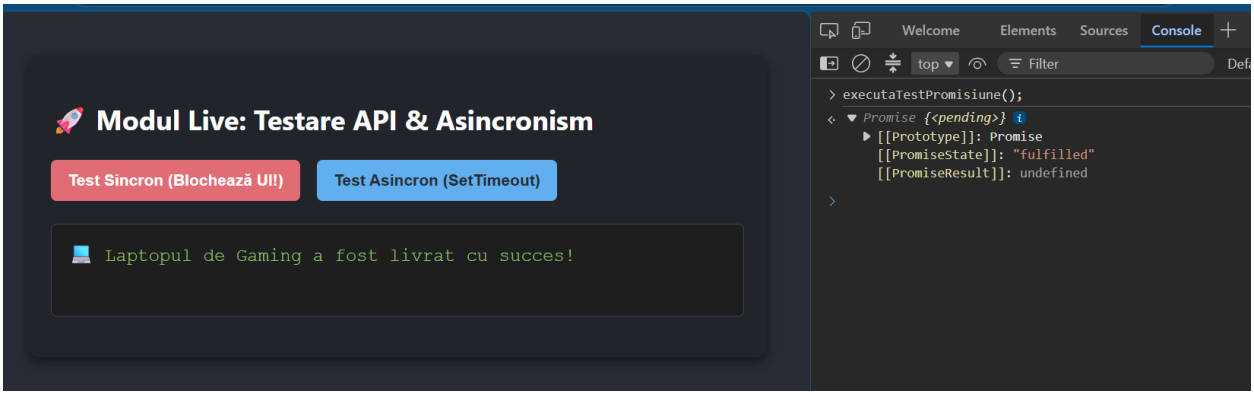
```
// 1. Creăm Promisiunea (Emulăm un apel de rețea care durează 2 secunde)
const comandaLaptop = () => {
    return new Promise((resolve, reject) => {
        setTimeout(() => {
            resolve("💻 Laptopul de Gaming a fost livrat cu succes!");
        }, 2000);
    });
};
```

```
// 2. Funcția de test modernă (Exact cum vom scrie în Playwright)
const executaTestPromisiune = async () => {
    ecran.innerHTML = "Aștept pachetul (Stare: Pending)...";

    // Cuvântul cheie 'await' pune pe PAUZĂ strict această funcție
    const rezultatAWB = await comandaLaptop();

    // Execuția continuă doar după Starea: Fulfilled
    ecran.innerHTML = rezultatAWB;
};
```





🟤 Pentru Capitolul 3: Prima cerere pe Rețea (Fetch GET)

🚀 **Scenariu Live:** Aducem date REALE de pe internet. Interogăm API-ul public JSONPlaceholder pentru a obține detaliile unui singur Utilizator și le injectăm în div-ul nostru ecran.

🤔 ❓ Cum testăm efectiv:

- Copiază funcția de mai jos în fișierul live-session.html, salvează și dă refresh.
- Mergi în Consola F12, scrie comanda verificaProfilUtilizator() și apasă Enter.
- Privește ecranul UI. Vei vedea pentru o fracțiune de secundă "Se contactează serverul...".

- Mesajul va fi înlocuit aproape instant de datele *reale* ale lui Leanne Graham (nume, email, website) care tocmai au fost descărcate live din baza de date JSONPlaceholder de pe internet.

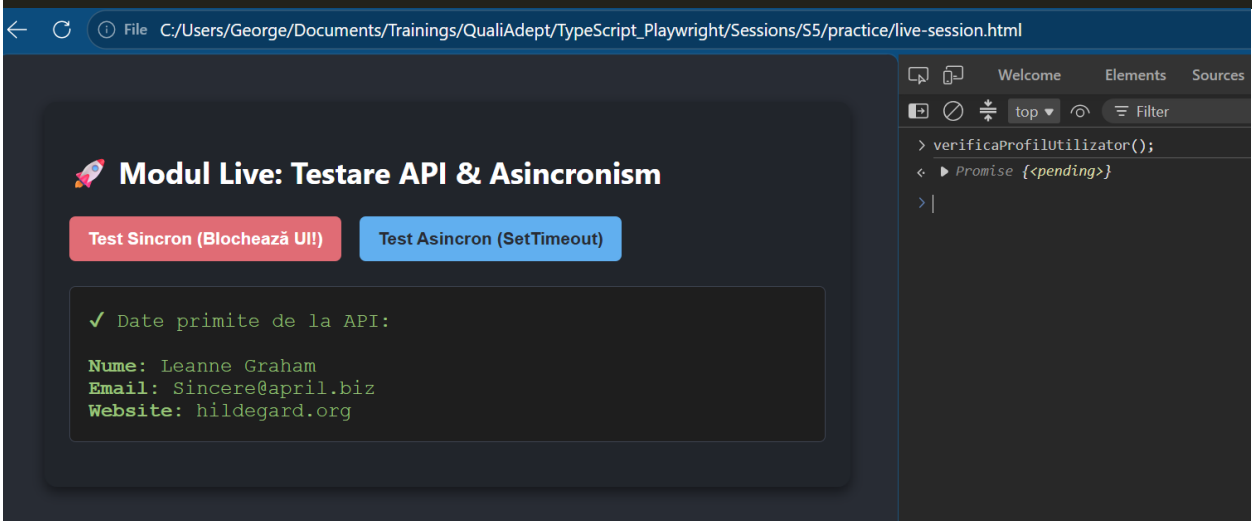
Cod de adăugat în tag-ul <script>:

```
const verificaProfilUtilizator = async () => {
    ecran.innerHTML = "Se contactează serverul (Fetch)...";

    // 1. Așteptăm "Chelnerul" să aducă Pachetul HTTP de la bucătărie
    const raspunsHTTP = await
fetch('https://jsonplaceholder.typicode.com/users/1');

    // 2. Așteptăm despachetarea textului în Obiect JavaScript (JSON)
    const dateUser = await raspunsHTTP.json();

    // 3. Validare & Randare pe ecranul nostru din HTML
    ecran.innerHTML = `
        ✓ Date primite de la API:<br><br>
        <strong>Nume:</strong> ${dateUser.name} <br>
        <strong>Email:</strong> ${dateUser.email} <br>
        <strong>Website:</strong> ${dateUser.website}
    `;
};
```



🟤 Pentru Capitolul 4 (Bonus): Trimiterea Datelor pe Server (Fetch POST)

🚀 **Scenariu Live:** Simulăm o comandă E2E de "Creare Cont" via API, ocolind interfața grafică. Setăm Metoda, Headers și Body, și primim un status de la server în interfața noastră.

🤯 ❓ **Cum testăm efectiv:**

- Adaugă funcția în live-session.html, salvează și dă refresh.
- În Consola F12, rulează creareTaskPrinAPI().
- Pe ecranul negru va apărea mesajul de trimitere, urmat rapid de mesajul verde de succes.
- Uită-te atent la ID-ul printat pe ecran: serverul îți va returna ID: 101, demonstrând practic că a primit, a procesat și a simulat adăugarea corectă a obiectului tău (payload-ului).

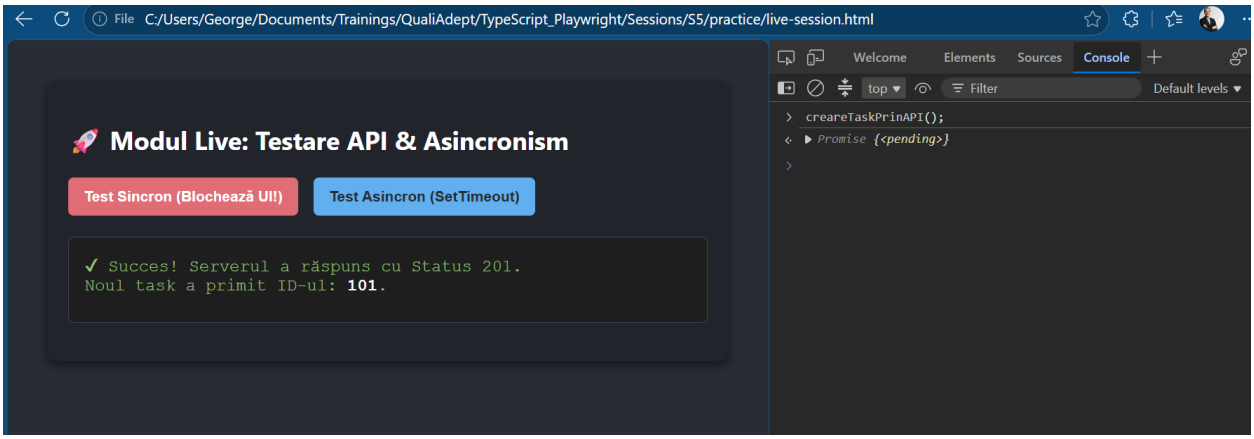
Cod de adăugat în tag-ul <script>:

```
const creareTaskPrinAPI = async () => {
    ecran.innerHTML = "Se trimit datele noi către baza de date...";

    // 1. Obiectul nostru JavaScript
    const payload = {
        title: "Învață Playwright API",
        body: "Testare directă de backend",
        userId: 99
    };

    // 2. Cererea de tip POST (Create)
    const raspuns = await
fetch('https://jsonplaceholder.typicode.com/posts', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(payload) // Traducem Obiectul JS în Text
pentru Internet
    });

    // 3. Extragem răspunsul salvat de server
    if (raspuns.status === 201) { // 201 = Created
        const dateSalvate = await raspuns.json();
        ecran.innerHTML = `✓ Succes! Serverul a răspuns cu Status 201. <br>
Noul task a primit ID-ul: <strong
style="color:white;">${dateSalvate.id}</strong>.`;
    }
};
```



🟤 Pentru Capitolul 5: End-to-End & Eșec Controlat (try/catch)

🚀 **Scenariu Live:** Aducem o listă de 3 elemente și le injectăm în elementul listaHtml (<ul>). Forțăm o eroare de rețea (URL greșit) pentru a vedea cum try/catch protejează aplicația de crash.

🤔 **Cum testăm efectiv:**

- Adaugă codul final, salvează și dă refresh.
- Rulează în Consola F12: testIncarcareLista().
- Observă cum scriptul rulează varianta cu URL_GRESIT. Interfața *nu se blochează*, ci afișează imediat textul roșu interceptat curat de blocul catch ("❌ EROARE CONTROLATĂ...").
- Acum, editează codul tău în VS Code: modifică linia fetch-ului să folosească parametrul valid: const raspuns = await fetch(URL_CORECT);.
- Salvează, dă refresh și rulează din nou testIncarcareLista() în consolă. Blocul try se execută de data asta cu succes, randând cele 3 elemente <li> sub formă de listă în UI!

Cod de adăugat în tag-ul <script>:

```
const testIncarcareLista = async () => {
    listaHtml.innerHTML = '';
    ecran.innerHTML = "Se încarcă lista...";

    try {
        // STRICĂM INTENȚIONAT URL-UL la varianta greșită
        const URL_CORECT =
'https://jsonplaceholder.typicode.com/todos?_limit=3';
        const URL_GRESIT =
```

```
'https://jsonplaceholder.typicode.com/todosXXX?_limit=3';

        // <--- Testează pe rând cu ambele URL-uri schimbând variabila de
mai jos
        const raspuns = await fetch(URL_GRESIT);

        if (!raspuns.ok) {
            throw new Error(`Serverul a răspuns cu eroarea fatală:
${raspuns.status}`);
        }

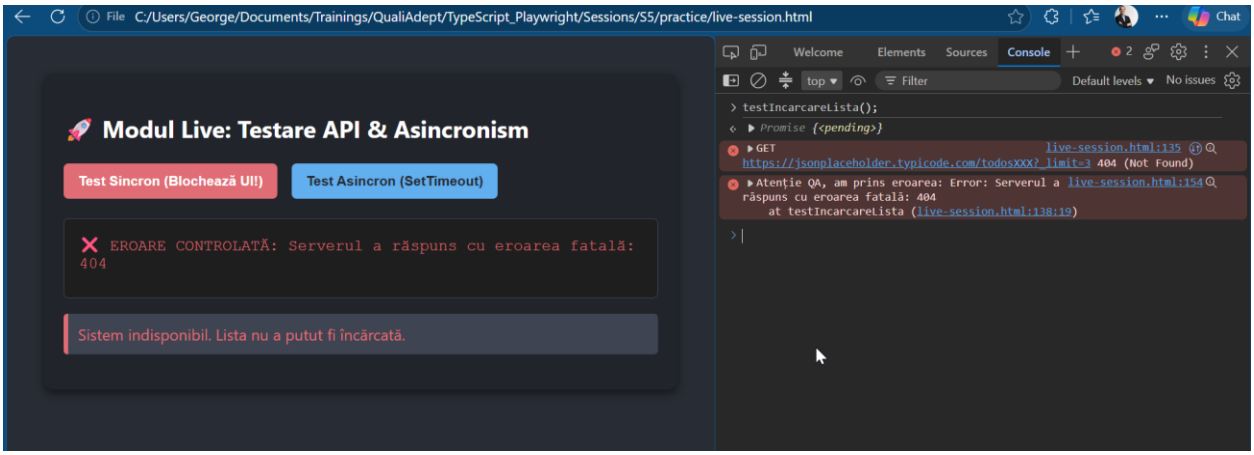
        const listaTaskuri = await raspuns.json();

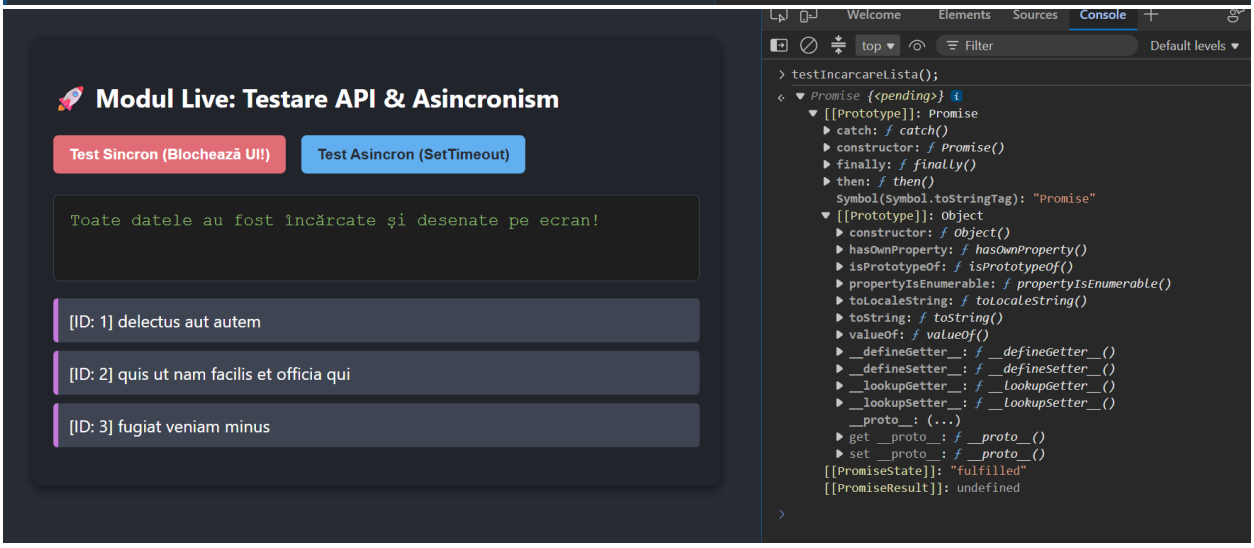
        // Randare în DOM folosind variabila listaHtml declarată la început
        listaTaskuri.forEach(task => {
            const li = document.createElement('li');
            li.textContent = `[ID: ${task.id}] ${task.title}`;
            listaHtml.appendChild(li);
        });

        ecran.innerHTML = "<span style='color:#98c379;'>Toate datele au
fost încărcate și desenate pe ecran!</span>";

    } catch (eroare) {
        // Aici ajungem dacă pică netul sau URL-ul e greșit
        console.error("Atenție QA, am prins eroarea:", eroare);

        ecran.innerHTML = `<span style='color:#e06c75;'>❌ EROARE
CONTROLATĂ: ${eroare.message}</span>`;
        listaHtml.innerHTML = "<li style='border-color: #e06c75; color:
#e06c75;'>Sistem indisponibil. Lista nu a putut fi încărcată.</li>";
    }
};
```





Workbook Avansat: Sesiunea 5 - JavaScript Asincron (Extra Practice)

📈 Acest material este conceput pentru aprofundarea individuală a conceptelor de asincronism, trecând dincolo de exercițiile de bază din timpul sesiunii live. Vom lucra pe un scenariu diferit: un "Dashboard de Monitorizare Servere".

Vei regăsi aici **8 exerciții practice**, incluzând concepte avansate (precum paralelizarea cu Promise.all()), pe care le vei executa direct în consolă pentru a manipula interfața aplicației.

Partea 1: Setup-ul Aplicației (Sandbox-ul)

Pasul 1: Salvează codul de mai jos într-un fișier numit **dashboard.html** folosind VS Code și deschide-l în Chrome.

Pasul 2: Toate cele 8 exerciții se vor rula introducând codul în **Consola (F12)**, observând cum reacționează UI-ul paginii.

```
<!DOCTYPE html>
<html lang="ro">
<head>
    <meta charset="UTF-8">
    <title>Server Monitoring Dashboard</title>
    <style>
        body { font-family: 'Segoe UI', Tahoma, sans-serif; background-
color: #1e1e2f; color: #fff; padding: 20px; }
        .dashboard { background: #2a2a40; padding: 25px; border-radius:
10px; box-shadow: 0 10px 20px rgba(0,0,0,0.3); max-width: 800px; margin:
auto; }
        .panel { background: #1e1e2f; border: 1px solid #444; border-
radius: 6px; padding: 15px; margin-bottom: 15px; }
        button { background: #00d2d3; color: #1e1e2f; border: none;
padding: 10px 20px; cursor: pointer; border-radius: 4px; font-weight: bold;
font-size: 14px; margin-right: 10px; transition: 0.2s;}
        button:hover { background: #01a3a4; }
        button.btn-danger { background: #ff6b6b; color: white; }
        #log-screen { font-family: 'Courier New', monospace; background:
#000; color: #10ac84; padding: 15px; min-height: 100px; border-radius: 4px;
margin-top: 15px; overflow-y: auto; max-height: 200px;}
        .server-grid { display: grid; grid-template-columns: 1fr 1fr; gap:
10px; }
        .server-card { padding: 10px; background: #3d3d5c; border-left: 5px
solid #5f27cd; border-radius: 4px; }
    </style>
</head>
<body>
    <div class="dashboard">
        <h2>📊 Server Monitor (QA Practice)</h2>
        <p>Apasă <strong>F12</strong>, deschide tab-ul
<strong>Console</strong> și rulează rând pe rând cele 8 exerciții.</p>
```

```
        <div class="panel">
            <button id="btn-stress" class="btn-danger">Stress Test
(Sincron)</button>
            <input type="text" id="test-input" placeholder="Încearcă să
scrii aici..." style="padding: 10px; width: 200px;">
        </div>

        <div class="panel">
            <h3>Terminal Loguri</h3>
            <div id="log-screen">Sistem pregătit. Aștept comenzi
asincrone...</div>
        </div>

        <div class="panel">
            <h3>Date Servere (Live API)</h3>
            <div id="server-data" class="server-grid">
                <!-- Datele aduse prin Fetch API vor fi injectate aici -->
            </div>
        </div>
    </div>

    <script>
        // Variabile globale ajutătoare pentru exercițiile tale
        const terminal = document.querySelector('#log-screen');
        const gridServere = document.querySelector('#server-data');

        // Funcție utilitară pentru a scrie ușor în terminalul negru
        const scrieInTerminal = (mesaj, culoare = '#10ac84') => {
            terminal.innerHTML += `<br><span style="color: ${culoare};">>>>
${mesaj}</span>`;
            terminal.scrollTop = terminal.scrollHeight; // Auto-scroll în
jos
        };
    </script>
</body>
</html>
```

Partea 2: Suita de Exerciții (Executate în Consola F12)

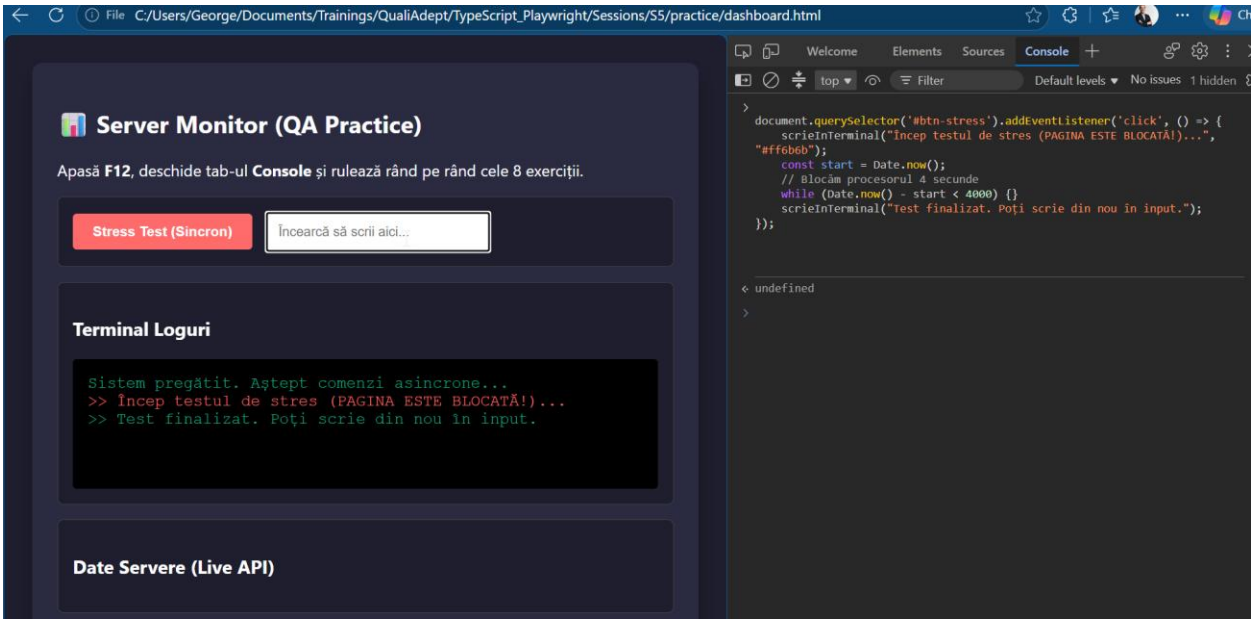
💻 Exercițiul 1: Coșmarul Sincron (Event Loop Blocked)

🚀 **Scenariu:** Demonstrăm cum o acțiune matematică intensă blochează complet pagina.

🤔 ❓ **Cum testăm efectiv:**

- Copiază codul de mai jos și rulează-l în consola F12 (apasă Enter).
- Imediat, du-te pe pagină și apasă fizic pe butonul roșu "Stress Test (Sincron)".
- Încearcă rapid să dai click în câmpul de text de lângă buton și să scrii ceva.
- Vei observa că browserul a "înghețat" complet timp de 4 secunde! Nu poți scrie nimic până nu se deblochează.

```
document.querySelector('#btn-stress').addEventListener('click', () => {
    scrieInTerminal("Încep testul de stres (PAGINA ESTE BLOCATĂ!)...",
"#ff6b6b");
    const start = Date.now();
    // Blocăm procesorul 4 secunde
    while (Date.now() - start < 4000) {}
    scrieInTerminal("Test finalizat. Poți scrie din nou în input.");
});
```



💻 Exercițiul 2: Înlănțuirea Asincronă (setTimeout)

🚀 **Scenariu:** Simulăm pașii unui test E2E (Setup -> Test -> Teardown) care rulează în fundal

fără să blocheze UI-ul.

🤔 ❓ **Cum testăm efectiv:**

- Copiază codul și rulează-l în consolă.
- Privește ecranul negru "Terminal Loguri" din pagină.
- În timp ce mesajele apar unul câte unul (la distanță de o secundă), dă click în input-ul alb și scrie text. Funcționează perfect, interfața nu e blocată deloc!

```
const ruleazaTestInFundal = () => {
    scrieInTerminal("1. [Async] Inițializare browser...");

    setTimeout(() => {
        scrieInTerminal("2. [Async] Navigare către pagină realizată (după
1s).", "#feca57");

        setTimeout(() => {
            scrieInTerminal("3. [Async] Element găsit și click efectuat
(după 2s).", "#feca57");

            setTimeout(() => {
                scrieInTerminal("4. [Async] Asertare trecută! Test
finalizat cu succes (după 3s).");
            }, 1000);

        }, 1000);

    }, 1000);
};

ruleazaTestInFundal();
```

💻 Exercițiul 3: Crearea unei Promisiuni de la Zero

Scenariu: Ne construim propriul mecanism "Promise" care simulează verificarea stării de sănătate a unui server (Health Check).

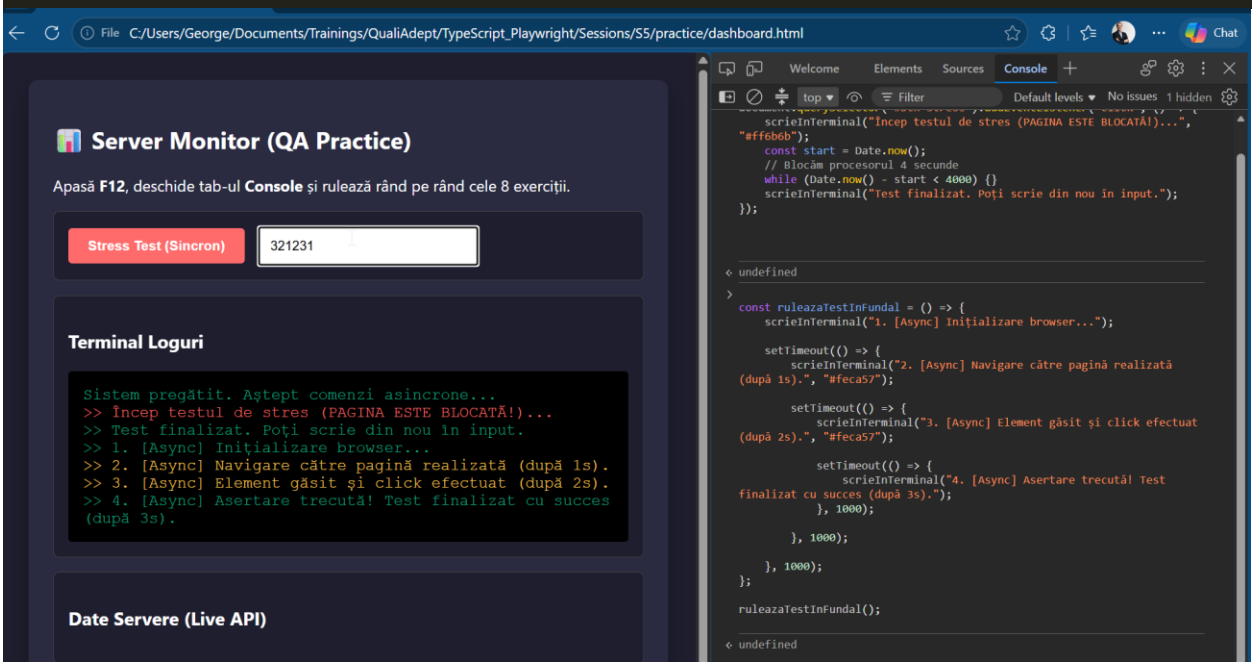
🤔 ❓ **Cum testăm efectiv:**

- Copiază și rulează codul în consolă.
- De data aceasta, uită-te la răspunsul printat de Consola F12 (nu în terminalul din pagină).
- Vei vedea un obiect Promise { <pending> }. Extinde-l (apasă pe săgețica de lângă el) după

ce au trecut 2 secunde. Vei vedea cum starea internă ([[PromiseState]]) i s-a schimbat în fulfilled.

```
const pingServer = () => {
    return new Promise((resolve, reject) => {
        setTimeout(() => {
            // Simulăm că serverul răspunde cu succes după 2 secunde
            resolve({ status: 200, mesaj: "Serverul DB este online și
rulează." });
        }, 2000);
    });
};

const promisiuneaMea = pingServer();
console.log("Starea promisiunii în acest moment:", promisiuneaMea);
```



💻 Exercițiul 4: Consumarea Promisiunii (async/await)

🚀 **Scenariu:** Acum folosim sintaxa Playwright (await) pentru a aștepta efectiv rezultatul Promisiunii create la Exercițiul 3 și a-l afișa în terminalul nostru.

🤔 ❓ Cum testăm efectiv:

- *Notă: Asigură-te că ai rulat deja Exercițiul 3 (ca funcția pingServer să existe în memorie).*

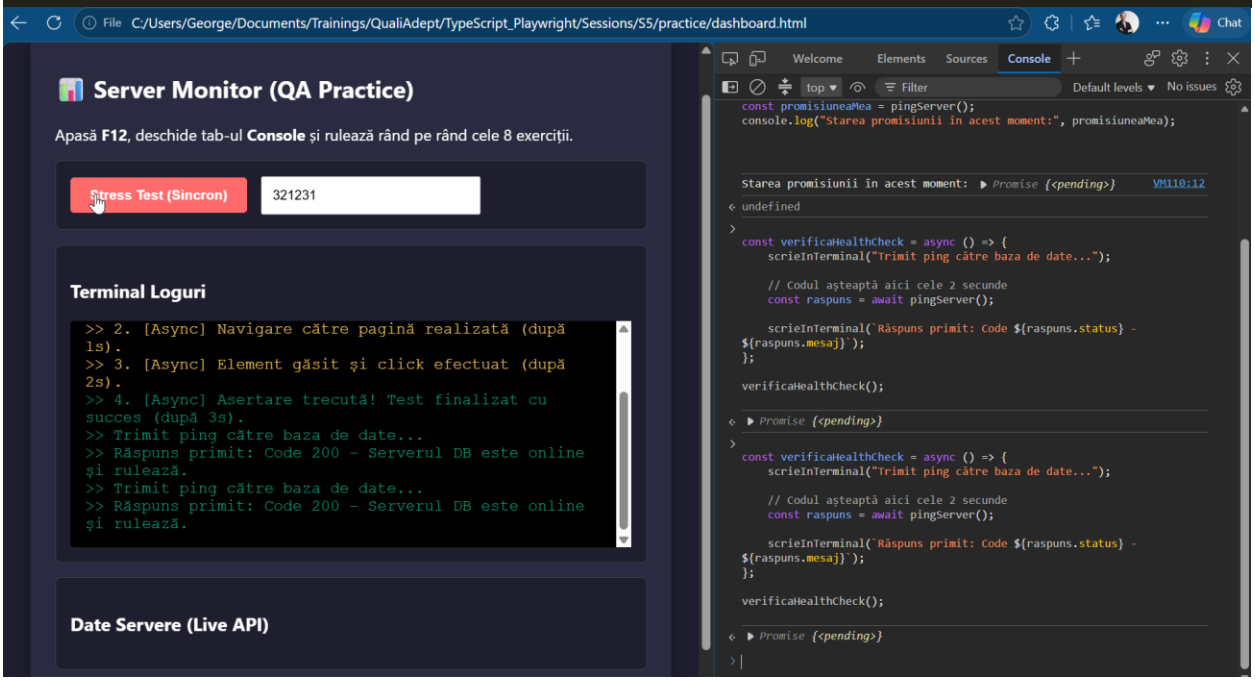
- Rulează codul de mai jos în consolă.
- Uită-te la terminalul negru din pagină. Va scrie "Trimit ping...", va aștepta exact 2 secunde pe pauză, apoi va printa răspunsul extras din Promisiune.

```
const verificaHealthCheck = async () => {
    scrieInTerminal("Trimit ping către baza de date...");

    // Codul așteaptă aici cele 2 secunde
    const raspuns = await pingServer();

    scrieInTerminal(`Răspuns primit: Code ${raspuns.status} -
${raspuns.mesaj}`);
};

verificaHealthCheck();
```



💻 Exercițiul 5: Fetch GET - Aducerea Datelor Reale

🚀 **Scenariu:** Folosim Fetch API pentru a aduce detaliile unui inginer QA de pe un server real de test (JSONPlaceholder) și îl afișăm sub formă de card HTML.

🤔 ❓ **Cum testăm efectiv:**

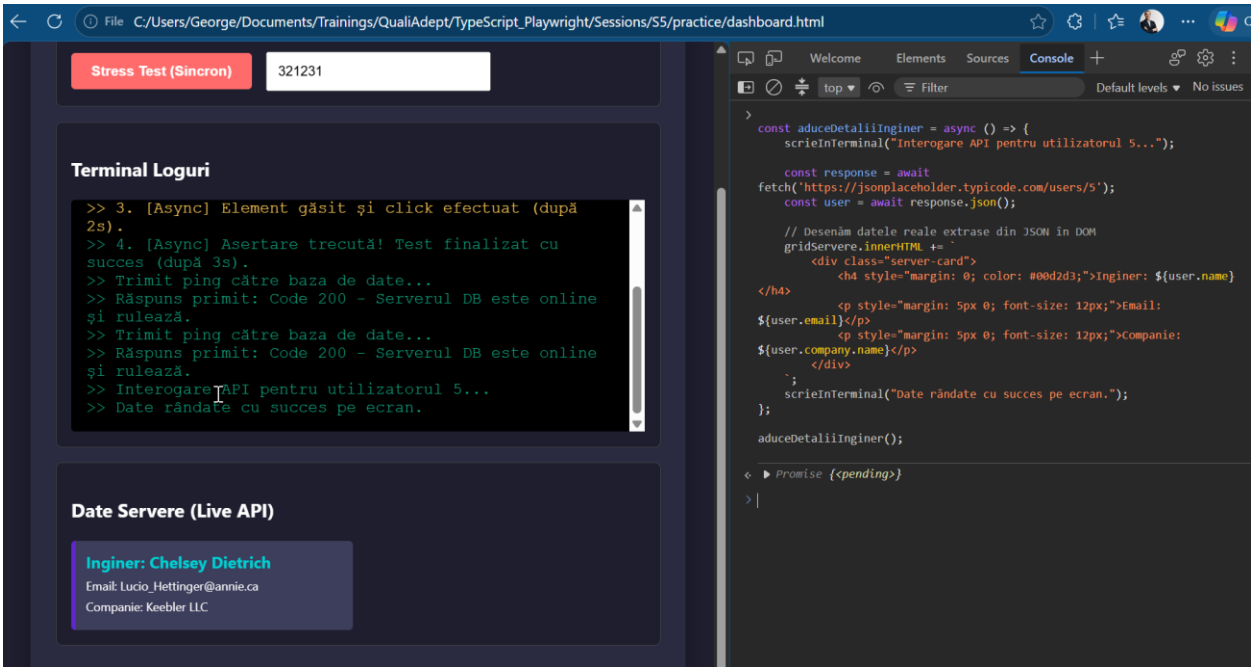
- Rulează codul în consolă.
- Privește secțiunea "Date Servere (Live API)" din dreapta-jos a paginii.
- Un card nou cu "Inginer: Chelsey Dietrich" va apărea instantaneu pe ecran, fiind date reale extrase chiar acum de pe internet!

```
const aduceDetaliiInginer = async () => {
    scrieInTerminal("Interogare API pentru utilizatorul 5...");

    const response = await
fetch('https://jsonplaceholder.typicode.com/users/5');
    const user = await response.json();

    // Desenăm datele reale extrase din JSON în DOM
    gridServere.innerHTML += `
        <div class="server-card">
            <h4 style="margin: 0; color: #00d2d3;">Inginer:
${user.name}</h4>
            <p style="margin: 5px 0; font-size: 12px;">Email:
${user.email}</p>
            <p style="margin: 5px 0; font-size: 12px;">Companie:
${user.company.name}</p>
        </div>
    `;
    scrieInTerminal("Date rândate cu succes pe ecran.");
};

aduceDetaliiInginer();
```



💻 Exercițiul 6: Fetch POST - Raportarea unui Bug pe rețea

🚀 **Scenariu:** Simulăm adăugarea unui Bug Report direct în baza de date prin API, configurând metoda, headerele și transformând obiectul JS în string (JSON.stringify).

🤔 **Cum testăm efectiv:**

- Rulează codul în consolă.
- Uită-te la terminalul negru de loguri.
- Vei vedea mesajul roz de succes care îți confirmă că serverul a procesat obiectul tău și i-a alocat noul ID: 101.

```
const raporteazaBugAPI = async () => {
    scrieInTerminal("Trimit Bug Report către server (POST)...");

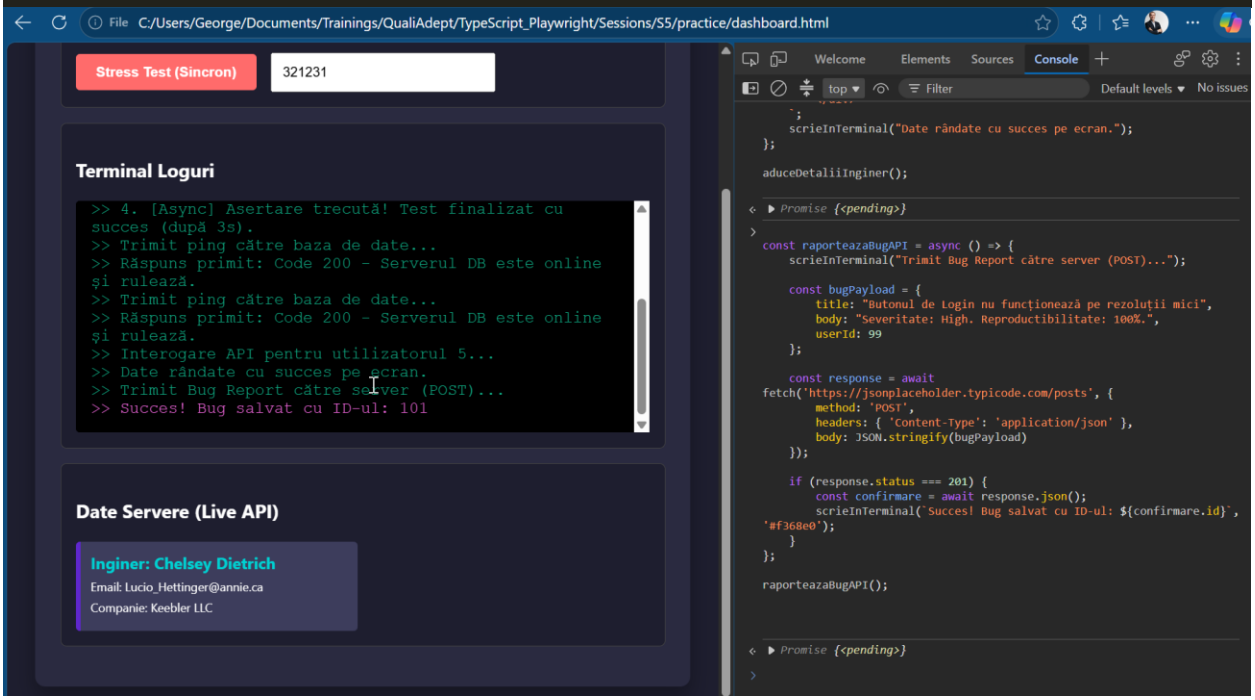
    const bugPayload = {
        title: "Butonul de Login nu funcționează pe rezoluții mici",
        body: "Severitate: High. Reproductibilitate: 100%.",
        userId: 99
    };

    const response = await
fetch('https://jsonplaceholder.typicode.com/posts', {
        method: 'POST',
```

```
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(bugPayload)
    });

    if (response.status === 201) {
        const confirmare = await response.json();
        scrieInTerminal(`Succes! Bug salvat cu ID-ul: ${confirmare.id}`,
'#f368e0');
    }
};

raporteazaBugAPI();
```



💻 Exercițiul 7: Reziliența cu Try/Catch

🚀 **Scenariu:** Ce se întâmplă dacă pică API-ul? Stricăm intenționat URL-ul. Dacă nu am avea try/catch, aplicația ar crăpa în tăcere. Noi interceptăm eroarea și afișăm un raport roșu.

🤔 **Cum testăm efectiv:**

- Rulează codul în consolă.
- Privește terminalul negru din pagină.
- Deoarece am stricat intenționat link-ul, codul va "sări" curat în blocul catch și va afișa mesajul

de eroare roșu. Pagina ta nu s-a blocat și totul e gestionat profesionist.

```
const testSigurDeRetea = async () => {
    try {
        scrieInTerminal("Inițializare test stabil...");

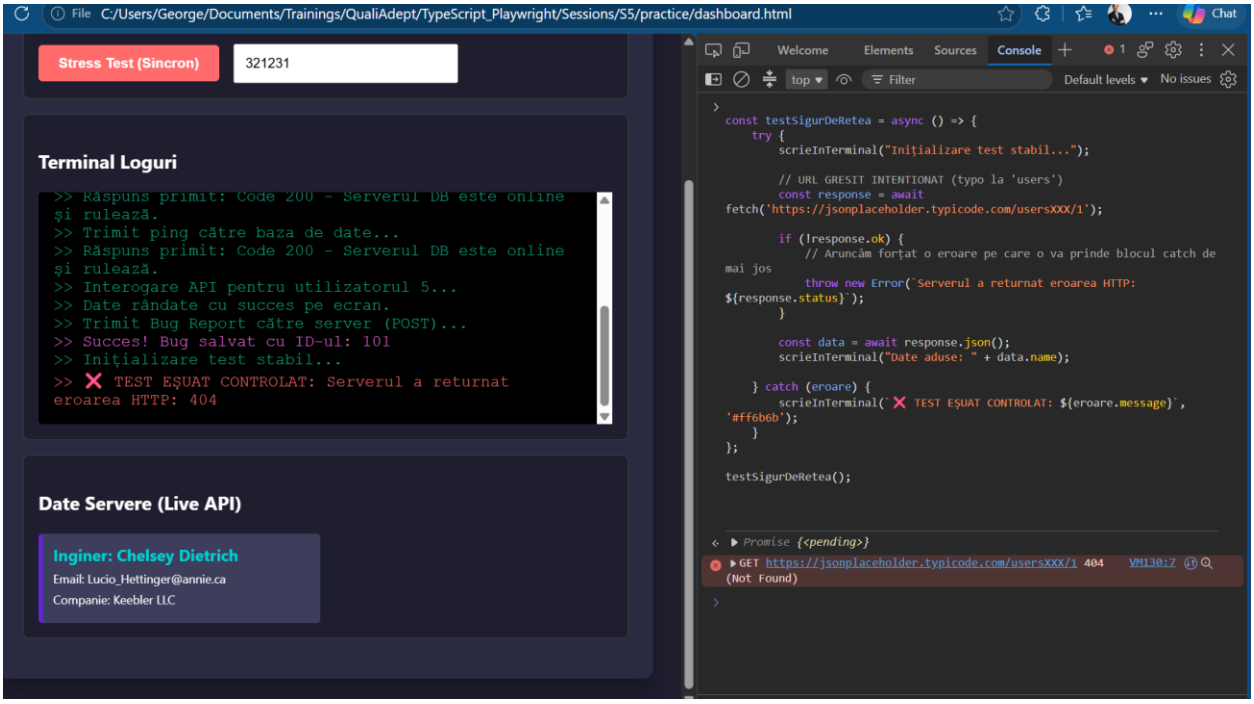
        // URL GRESIT INTENTIONAT (typo la 'users')
        const response = await
fetch('https://jsonplaceholder.typicode.com/usersXXX/1');

        if (!response.ok) {
            // Aruncăm forțat o eroare pe care o va prinde blocul catch de
mai jos
            throw new Error(`Serverul a returnat eroarea HTTP:
${response.status}`);
        }

        const data = await response.json();
        scrieInTerminal("Date aduse: " + data.name);

    } catch (eroare) {
        scrieInTerminal(`❌ TEST EȘUAT CONTROLAT: ${eroare.message}`,
'#ff6b6b');
    }
};

testSigurDeRetea();
```



💻 Exercițiul 8: Concept Avansat - Paralelizarea (Promise.all)

🚀 **Scenariu de nivel Senior:** Ai de verificat 3 elemente de pe 3 adrese diferite. Dacă le aștepți pe rând cu await, durează mult. Cu Promise.all le trimiți pe toate 3 pe rețea în același timp!

🤔 **Cum testăm efectiv:**

- Rulează codul în consolă.
- Privește atent secțiunea "Date Servere" și terminalul negru.
- Observă că, deși am cerut date pentru 3 utilizatori total diferiți, toate cele 3 carduri apar pe ecran EXACT în același timp, demonstrând execuția super-rapidă în paralel!

```
const descarcaMasaDeDate = async () => {
    scrieInTerminal("Lansăm 3 cereri de rețea SIMULTAN (Promise.all)...", "#feca57");

    try {
        // Pregătim cele 3 Promisiuni de Fetch (nu punem await în fața lor încă!)
        const cererea1 = fetch('https://jsonplaceholder.typicode.com/users/1').then(r => r.json());
        const cererea2 = fetch('https://jsonplaceholder.typicode.com/users/2').then(r => r.json());
```

```
        const cererea3 =
fetch('https://jsonplaceholder.typicode.com/users/3').then(r => r.json());

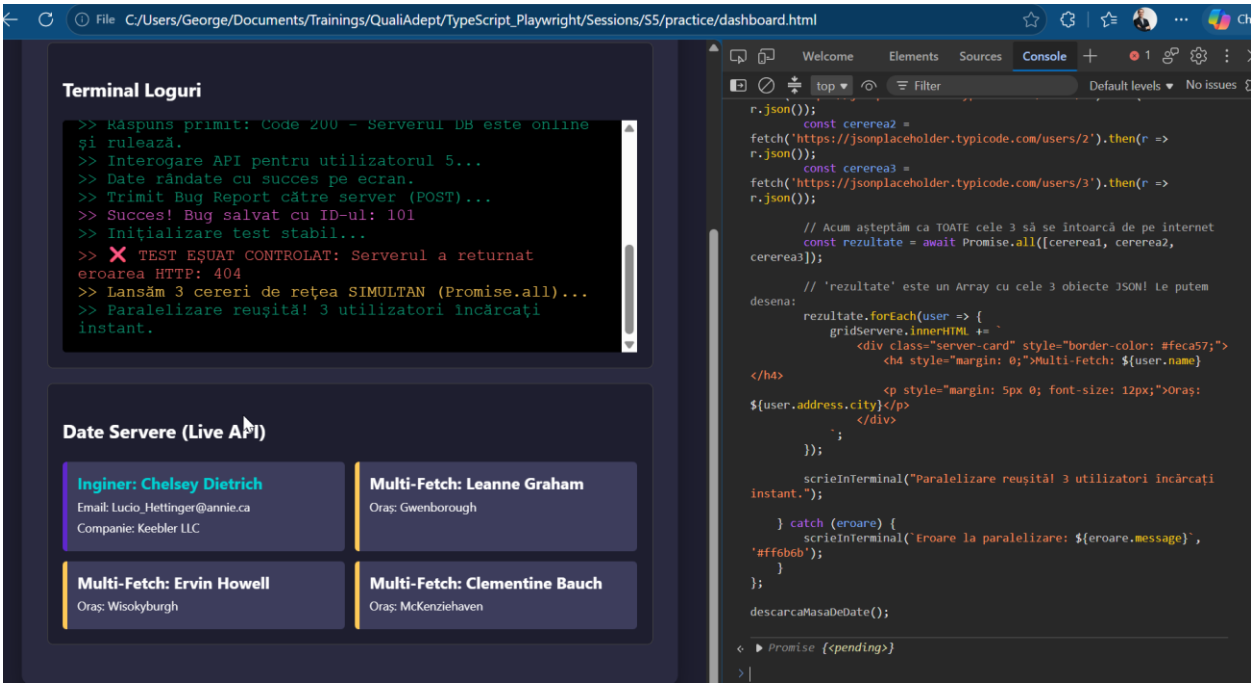
        // Acum așteptăm ca TOATE cele 3 să se întoarcă de pe internet
        const rezultate = await Promise.all([cererea1, cererea2,
cererea3]);

        // 'rezultate' este un Array cu cele 3 obiecte JSON! Le putem
desena:
        rezultate.forEach(user => {
            gridServere.innerHTML += `
                <div class="server-card" style="border-color: #feca57;">
                    <h4 style="margin: 0;">Multi-Fetch: ${user.name}</h4>
                    <p style="margin: 5px 0; font-size: 12px;">Oraș:
${user.address.city}</p>
                </div>
            `;
        });

        scrieInTerminal("Paralelizare reușită! 3 utilizatori încărcați
instant.");

    } catch (eroare) {
        scrieInTerminal(`Eroare la paralelizare: ${eroare.message}`,
'#ff6b6b');
    }
};

descarcaMasaDeDate();
```


File C:/Users/George/Documents/Trainings/QualiAdept/TypeScript_Playwright/Sessions/S5/practice/dashboard.html
Terminal Loguri
>> Răspuns primit: Code 200 - Serverul DB este online şi rulează.
>> Interogare API pentru utilizatorul 5...
>> Date rândate cu succes pe ecran.
>> Trimit Bug Report către server (POST)...
>> Succes! Bug salvat cu ID-ul: 101
>> Inițializare test stabil...
>> ❌ TEST EŞUAT CONTROLAT: Serverul a returnat eroarea HTTP: 404
>> Lansăm 3 cereri de rețea SIMULTAN (Promise.all)...
>> Paralelizare reuşită! 3 utilizatori încărcați instant.
Date Servere (Live API)
Inginer: Chelsey Dietrich
Email: Lucio_Hettinger@annie.ca
Companie: Keebler LLC
Multi-Fetch: Leanne Graham
Oraș: Gwenborough
Multi-Fetch: Ervin Howell
Oraș: Wisokyburgh
Multi-Fetch: Clementine Bauch
Oraș: McKenziehaven
Welcome
Elements
Sources
Console
top
Filter
Default levels
No issues
r.json());
const cererea2 = fetch('https://jsonplaceholder.typicode.com/users/2').then(r => r.json());
const cererea3 = fetch('https://jsonplaceholder.typicode.com/users/3').then(r => r.json());
// Acum aşteptăm ca TOATE cele 3 să se întoarcă de pe internet
const rezultate = await Promise.all([cererea1, cererea2, cererea3]);
// 'rezultate' este un Array cu cele 3 obiecte JSON! Le putem desena:
rezultate.forEach(user => {
gridServere.innerHTML += `

<h4 style="margin: 0;">Multi-Fetch: ${user.name}</h4>
Oraș: ${user.address.city}

`;
});
scrieInTerminal("Paralelizare reuşită! 3 utilizatori încărcați instant.");
} catch (eroare) {
scrieInTerminal(`Eroare la paralelizare: ${eroare.message}`, '#ff6b6b');
}
};
descarcaMasaDeDate();
Promise {<pending>}